



UNIVERSIDAD DE MURCIA

FACULTAD DE INFORMÁTICA

App de trading automático con modelos computacionales

Trabajo de Fin de Grado

Autor:

Alejandro López López
Grado en Ingeniería Informática

Tutores:

Dra. Doña Raquel Martínez España
Dr. D. José Tomás Palma Mendez

Índice general

Índice de figuras	4
Índice de tablas	5
1. Resumen	6
2. Extended Abstract	7
3. Introducción	8
4. Objetivos	10
4.1. Objetivo General	10
4.2. Objetivos Específicos	10
5. Estado del Arte	11
5.1. Evolución del Trading Algorítmico	11
5.2. Datos y Modelos Predictivos	11
5.3. Arquitecturas de Software para Sistemas de Trading	13
5.4. Posicionamiento del Trabajo	13
6. Análisis y Metodología	15
6.1. Vinculación con la Problemática Inicial	15
6.2. Metodología de Desarrollo	15
6.2.1. Incrementos de Implementación	16
6.3. Tecnologías y Herramientas Utilizadas	18
6.3.1. Librerías Críticas	18
6.4. Arquitectura General	21
6.4.1. Estructura y componentes del sistema	21
6.4.2. Comunicación entre procesos	25

6.4.3.	Ventajas y desventajas de la arquitectura	26
6.5.	Indicadores Técnicos Implementados	26
6.5.1.	La Estrategia Base del Sistema	27
6.5.2.	Indicador: Media Móvil Simple (SMA)	27
6.5.3.	Indicador: Media Móvil Exponencial (EMA)	28
6.5.4.	Indicador: Relative Strength Index (RSI)	29
6.5.5.	Indicador: Moving Average Convergence Divergence (MACD)	30
6.5.6.	Indicador: <i>Z-score</i> de Volumen	30
6.5.7.	Indicador: ATR Normalizado	31
6.5.8.	Indicador: Bandas de Bollinger	31
6.5.9.	Indicador: Tasa de Cambio (ROC)	31
6.5.10.	Relación entre Indicadores y Uso Combinado	32
6.5.11.	Período de Calentamiento	32
6.6.	Módulo de Inteligencia Artificial y Validación Experimental	33
6.6.1.	Ingeniería de características y generación de etiquetas	33
6.6.2.	Preparación del conjunto de datos	34
6.6.3.	Algoritmos soportados y optimización de hiperparámetros	34
6.7.	Diagramas de Flujo	35
6.7.1.	Gestión de Usuarios y Acceso	36
6.7.2.	Gestión de Capital (<i>Wallets</i>)	37
6.7.3.	Obtención de Datos (ETL)	38
6.7.4.	Gestión de Estrategias	39
6.7.5.	Backtesting	42
6.7.6.	Trading en Tiempo Real	44
6.7.7.	Entrenamiento de Modelos	46
6.7.8.	Optimización de Hiperparámetros	48
7.	Desarrollo del Trabajo	52
7.1.	Introducción	52
7.2.	Trazabilidad de Objetivos y Evidencia	52
7.3.	Manual de usuario	54
7.4.	Resultados	58
7.4.1.	Cobertura de tests automáticos	58
8.	Conclusiones y Vías Futuras	61

A. Glosario	62
B. Esquema de la base de datos	72
C. Referencia del protocolo IPC	78
D. Espacios de búsqueda de hiperparámetros	80

Índice de figuras

6.1. Esquema conceptual de la orientación hexagonal del proyecto; no representa literalmente toda la jerarquía de paquetes.	22
6.2. Flujo de autenticación (login/signup) y ayuda	37
6.3. Proceso de creación y listado de <i>wallets</i>	38
6.4. Flujo de comandos de listado (ls, le, lsa)	40
6.5. Ciclo de vida: start, stop y term	42
6.6. Flujo de ejecución del motor de <i>backtesting</i>	44
6.7. Flujo de <i>trading</i> en tiempo real y comunicación Java-Python	46
6.8. Flujo de entrenamiento de modelos supervisados	48
6.9. Flujo de optimización de hiperparámetros	51

Índice de cuadros

7.1. Trazabilidad de objetivos específicos con evidencia del desarrollo	53
7.2. Referencia de comandos: cuenta, cartera, datos y simulación histórica	56
7.3. Referencia de comandos: negociación en tiempo real e inteligencia artificial	57
7.4. Resumen de suites de tests automáticos	59
7.5. Cobertura de tests por módulo funcional	60
B.1. Columnas de la tabla <code>usuario</code>	72
B.2. Columnas de la tabla <code>vela</code>	73
B.3. Columnas de la tabla <code>wallet</code>	74
B.4. Columnas de la tabla <code>instancia_estrategia</code>	75
B.5. Columnas de la tabla <code>instancia_simbolos</code>	75
B.6. Columnas de la tabla <code>posicion</code>	76
B.7. Columnas de la tabla <code>ledger_entry</code>	76
B.8. Columnas de la tabla <code>indicador_tecnico</code>	76
B.9. Columnas de la tabla <code>capital_reservado</code>	77
C.1. Campos del objeto envoltante <code>IpcEnvelope</code>	78
C.2. Tipos de mensaje del protocolo IPC, agrupados por dominio.	79
C.3. Códigos de salida de los motores Python.	79
D.1. Rangos de búsqueda de hiperparámetros por familia de modelo	80

1. Resumen

Texto comentado

2. Extended Abstract

Texto comentado

3. Introducción

Las criptomonedas han revolucionado el panorama financiero, ofreciendo nuevas oportunidades de inversión y especulación a corto, mediano y largo plazo (Baur y col., 2017). Sin embargo, la volatilidad inherente a estos activos y la complejidad de los mercados digitales presentan desafíos significativos para los inversores (Katsiampa, 2017). A diferencia de los mercados bursátiles tradicionales, los mercados de criptomonedas operan de forma ininterrumpida, los siete días de la semana, a escala global y sin un regulador centralizado. Esta naturaleza descentralizada, sumada a la velocidad con la que los precios pueden variar en cuestión de minutos, hace que la toma de decisiones humana por sí sola sea insuficiente para competir con los sistemas automatizados que dominan actualmente estos mercados. Mientras que en la bolsa tradicional un inversor minorista podía analizar la situación con calma durante el horario de mercado, en el ecosistema cripto esta ventana se ha cerrado: el mercado no duerme, las oportunidades son efímeras y los errores de juicio tienen consecuencias inmediatas. A esto se añade que la barrera de entrada se ha reducido drásticamente gracias a plataformas de fácil acceso, lo que ha atraído a millones de inversores sin experiencia previa que se enfrentan a un entorno hostil sin las herramientas ni los conocimientos necesarios para navegarlo de forma racional.

La necesidad de este proyecto radica, en primer lugar, en la dificultad inherente de los usuarios para reaccionar en tiempo real ante estas rápidas fluctuaciones del mercado. A esto se suma una carencia generalizada de educación financiera básica, especialmente entre los jóvenes y los inversores minoristas (Lusardi & Mitchell, 2014). En la actualidad, resulta complejo iniciarse en el mundo de la inversión de forma segura y sin arriesgar capital real. Este problema se ve agravado por la sobreabundancia de ruido mediático y atención pública desmedida en torno a estos activos. Como demuestra la literatura reciente, esto fomenta expectativas irreales que empujan a los usuarios inexpertos (especialmente a los más jóvenes) a operar de forma altamente especulativa y con excesiva frecuencia, lo que habitualmente desemboca en menores rendimientos y la pérdida paulatina de sus ahorros (Hasso y col., 2019). Por otro lado, el trading algorítmico ofrece la gran ventaja de eliminar el sesgo emocional de las decisiones de inversión, aportando una mayor disciplina y consistencia matemática en la ejecución de las estrategias (Chan, 2013). Sin embargo, la alta barrera de entrada, derivada de la profunda complejidad técnica y la falta de acceso a herramientas de simulación adecuadas, dificulta enormemente el aprendizaje (Treleaven y col., 2013).

En este contexto, las *estrategias de trading* constituyen la respuesta natural a estos problemas. Una estrategia no es más que un conjunto de reglas objetivas y reproducibles basadas en indicadores técnicos como la media móvil, el RSI o el MACD— que determi-

nan de forma autónoma cuándo comprar y cuándo vender un activo, eliminando así la subjetividad y la reacción emocional del inversor humano (Murphy, 1999). El valor real de las estrategias, sin embargo, no reside únicamente en su ejecución en tiempo real, sino en la posibilidad de validarlas previamente sobre datos históricos mediante el *backtesting*: un proceso que permite medir su rendimiento pasado antes de comprometer capital real. Esta capacidad de experimentación controlada resulta fundamental para el aprendizaje, pues permite al inversor iterar sobre sus hipótesis, detectar sus errores y desarrollar un criterio crítico sin consecuencias económicas. A pesar de ello, el diseño, la implementación y la validación de estas estrategias sigue siendo una tarea reservada a perfiles con conocimientos avanzados de programación y matemáticas financieras, lo que excluye a la inmensa mayoría de los inversores interesados en adoptarlas.

Para abordar esta problemática, este proyecto se centra en el desarrollo de una plataforma de trading algorítmico orientada a reducir esta brecha técnica, democratizando el acceso al trading cuantitativo. El sistema proporciona a los estudiantes un entorno de simulación seguro (*Paper Trading*) donde pueden experimentar, aprender de sus errores y validar sus hipótesis sin riesgo financiero. Además, la plataforma centraliza la exportación de resultados en formato CSV para su auditoría. Asimismo, para aquellos con un perfil más técnico, el sistema proporciona una base sólida para la recopilación de datos históricos de múltiples monedas y la aplicación práctica de modelos de Inteligencia Artificial, ya sea entrenando algoritmos con indicadores básicos o retroalimentándolos con los resultados de las estrategias del usuario.

Finalmente, para facilitar la curva de aprendizaje, el formato de las estrategias se basa en el estándar utilizado por *Freqtrade* (Freqtrade Development Team, 2023), una plataforma de código abierto escrita en Python. Este formato permite a los usuarios definir sus propias reglas operativas de forma intuitiva. Las estrategias se implementan como scripts que interactúan con el motor de cálculo para recibir datos de mercado, procesar señales y enviar órdenes, permitiendo al estudiante centrarse puramente en la lógica financiera mientras el sistema gestiona la complejidad subyacente.

4. Objetivos

4.1. Objetivo General

El objetivo principal de este Trabajo de Fin de Grado es desarrollar una plataforma de trading algorítmico y simulación (backtesting) para el mercado de criptomonedas, capaz de integrar dinámicamente indicadores técnicos personalizados y modelos de Inteligencia Artificial para la predicción direccional del precio.

4.2. Objetivos Específicos

Para alcanzar el objetivo general propuesto, se establecen los siguientes objetivos específicos, alineados con las fases del ciclo de vida del desarrollo de software: Estos objetivos se derivan de la problemática descrita en el Capítulo 3 y estructuran la validación técnica posterior en los Capítulos 6, 7 y 8.

- **Estudiar** el estado del arte del trading cuantitativo, analizando la formulación matemática de los indicadores técnicos básicos y los paradigmas actuales de *Machine Learning* aplicados a series temporales financieras.
- **Analizar** los requisitos funcionales y no funcionales necesarios para la construcción de un sistema financiero de alta concurrencia.
- **Diseñar** una arquitectura de software modular y distribuida que desacople la lógica de orquestación transaccional del motor de entrenamiento e inferencia estadística.
- **Desarrollar** un motor de ingesta y sincronización de datos de mercado automatizado capaz de procesar, limpiar y almacenar masivamente velas transaccionales (*candlesticks*) desde fuentes externas (APIs).
- **Implementar** un pipeline de Inteligencia Artificial paramétrico que permita al usuario configurar, entrenar y evaluar múltiples algoritmos predictivos inyectando hiperparámetros de forma dinámica.
- **Evaluar y validar** la robustez y efectividad de la plataforma mediante la ejecución de simulaciones históricas (*backtesting*), analizando métricas de rendimiento computacional y financiero (tasa de acierto, factor de beneficio y reducción máxima o *drawdown*).

5. Estado del Arte

El desarrollo de sistemas de trading algorítmico ha experimentado una transformación radical en la última década, pasando de la ejecución de simples reglas estáticas a la integración de modelos complejos de Inteligencia Artificial (IA) (Treleaven y col., 2013). En esta sección se analizan las tecnologías, paradigmas y algoritmos que conforman el estado actual de la predicción de mercados financieros y las arquitecturas de software de trading. El recorrido de este capítulo sigue la misma lógica del sistema desarrollado: primero se sitúa la evolución del trading algorítmico, después cómo se construyen las variables de entrada del módulo predictivo, a continuación se revisan las familias de modelos que permiten formular la predicción direccional y, finalmente, se analiza la arquitectura software necesaria para integrar cálculo analítico, persistencia y ejecución.

5.1. Evolución del Trading Algorítmico

El trading algorítmico consiste en el uso de programas informáticos para introducir órdenes de compra y venta en los mercados financieros con base en un conjunto de reglas predefinidas (Chan, 2013). Históricamente, estos sistemas se basaban en el Análisis Técnico tradicional, utilizando indicadores matemáticos retrospectivos (Murphy, 1999). Sin embargo, la Hipótesis de los Mercados Eficientes (Fama, 1970) sugiere que los precios de los activos reflejan toda la información disponible, lo que hace que los métodos puramente lineales sean insuficientes en mercados de alta liquidez como el de las criptomonedas (Katsiampa, 2017). Esto ha provocado la migración hacia el análisis cuantitativo y el modelo no lineal mediante aprendizaje automático (López de Prado, 2018).

Esta evolución explica que, antes de discutir algoritmos concretos, resulte necesario revisar cómo se construyen las variables con las que esos modelos operan. En un sistema como el desarrollado en este trabajo, la predicción no se plantea sobre velas OHLCV en bruto, sino sobre indicadores que representan la tendencia, momento y volatilidad.

5.2. Datos y Modelos Predictivos

A pesar del auge de la IA, los indicadores técnicos clásicos siguen siendo la base fundamental para la fase de extracción de características (López de Prado, 2018). La literatura actual demuestra que la combinación de indicadores de tendencia (como la Media Móvil Exponencial, EMA), de momento (como el Relative Strength Index, RSI) y

de volatilidad (como el Average True Range, ATR) proporcionan a los modelos predictivos el contexto necesario para identificar regímenes de mercado (Murphy, 1999, Caps. 9, 10 y 16). El preprocesamiento de estos datos en ventanas temporales escalonadas es vital para reducir el ruido del precio (Chan, 2013; López de Prado, 2018).

En este proyecto, el módulo predictivo se apoya precisamente en un conjunto de variables calculables y comparables. Una vez construido ese espacio de características, el problema pasa a ser decidir qué familias de modelos supervisados resultan más adecuadas para predecir la dirección de la siguiente vela. La predicción del movimiento direccional del precio (clasificación: sube, baja o lateral) es uno de los problemas más complejos en el ámbito del aprendizaje automático, debido a la baja relación señal-ruido de los mercados financieros (Fama, 1970). Actualmente, la industria se apoya en las siguientes familias de algoritmos:

Modelos de Ensamblado basados en Árboles. Los métodos de ensamblado dominan las competiciones de ciencia de datos tabulares financieras (López de Prado, 2018). **Random Forest** actúa como el estándar de la industria para establecer referencias robustas gracias a su resistencia al sobreajuste, lograda al combinar múltiples árboles de decisión independientes (Breiman, 2001). Por su parte, los métodos de **Gradient Boosting** como XGBoost (Chen & Guestrin, 2016) y LightGBM (Ke y col., 2017) destacan por su alta precisión en conjuntos de datos tabulares: XGBoost incorpora técnicas de regularización que reducen el sobreajuste y mejoran la selección de variables relevantes (Chen & Guestrin, 2016), mientras que LightGBM ofrece un entrenamiento más rápido y eficiente en memoria (Ke y col., 2017), siendo el algoritmo preferido para procesar grandes volúmenes de datos históricos.

Máquinas de Vectores de Soporte (SVM). En su formulación matemática, las SVM transforman los datos de entrada a un espacio de mayor dimensión mediante funciones núcleo para encontrar un hiperplano de separación óptimo (Cortes & Vapnik, 1995). Aplicado al dominio financiero, aunque estas técnicas resultan computacionalmente costosas al procesar conjuntos de datos masivos, siguen siendo una herramienta valiosa para identificar cambios de tendencia macroeconómicos cuando se utilizan con variables muy refinadas (Pedregosa y col., 2011; Treleaven y col., 2013).

Redes Neuronales y Aprendizaje Profundo. Las redes neuronales profundas han demostrado una capacidad excepcional para aprender representaciones no lineales complejas en datos financieros (Goodfellow y col., 2016). Cuando el problema se formula como clasificación direccional sobre un conjunto de indicadores técnicos precalculados, las arquitecturas densas totalmente conectadas resultan la elección natural: el modelo recibe los indicadores como entrada y predice la dirección de la siguiente vela (Goodfellow y col., 2016). La arquitectura típica combina capas de neuronas con funciones de activación no lineales, capas de regularización para reducir el sobreajuste y una capa de salida para clasificación multiclase. Su principal ventaja sobre los modelos de ensamblado radica en la capacidad de gestionar interacciones no lineales de alto orden entre indicadores (Goodfellow y col., 2016); sin embargo, su mayor tendencia al sobreajuste en conjuntos de datos reducidos y su coste computacional de entrenamiento los convierten en una herramienta complementaria a los métodos basados en árboles (López de Prado, 2018).

5.3. Arquitecturas de Software para Sistemas de Trading

Una vez fijados el tipo de variables y las familias de modelos predictivos, la cuestión deja de ser solo algorítmica y pasa a ser también de integración: cómo combinar entrenamiento, inferencia, ingesta de datos y ejecución transaccional sin mezclar responsabilidades. Ese es el contexto en el que la arquitectura software adquiere relevancia para este trabajo.

El ecosistema del desarrollo de bots de trading requiere una arquitectura distribuida que soporte alta concurrencia y baja latencia (Treleaven y col., 2013). El estado del arte en ingeniería de software financiera divide estas soluciones en dos capas (Richards, 2022):

1. **Capa Transaccional y de Orquestación:** Generalmente desarrollada en lenguajes fuertemente tipados y compilados orientados a concurrencia, como Java con entornos como Spring Boot (VMware Tanzu & Spring Team, 2024) o C++. Esta capa gestiona el enrutamiento de red, la persistencia masiva en bases de datos (inserciones masivas para series temporales) (Kimball & Caserta, 2004) y la ingesta de datos en tiempo real (Richards, 2022).
2. **Capa Analítica (Inferencia y Entrenamiento):** Implementada casi en su totalidad en Python (Hilpisch, 2018) debido a su ecosistema superior en ciencia de datos.

Dos de los problemas más comunes y críticos en este tipo de arquitecturas son la comunicación entre capas y la configuración de los modelos predictivos. Por un lado, la comunicación entre procesos (IPC) representa uno de los mayores desafíos técnicos (Furuhashi, 2013): las soluciones actuales optan por colas de mensajes, servicios web o comunicación directa mediante tuberías y entrada/salida estándar (Kleppmann, 2017; Newman, 2015; Treleaven y col., 2013). Por otro lado, ajustar los parámetros de los modelos predictivos de forma manual resulta inviable cuando el espacio de posibilidades es amplio; por ello, el estado del arte emplea métodos de optimización bayesiana que guían la búsqueda de forma eficiente (López de Prado, 2018), reduciendo el tiempo necesario sin sacrificar la calidad de la configuración encontrada (Feurer & Hutter, 2019).

5.4. Posicionamiento del Trabajo

Sobre esta base, el posicionamiento del trabajo puede formularse con mayor precisión: no se trata solo de construir otro bot de trading, sino de proponer una plataforma que combine el estado del arte en variables técnicas, modelos predictivos y desacoplamiento arquitectónico dentro de un entorno docente y reproducible.

Para comprender el valor de la plataforma desarrollada en este proyecto, es necesario analizar las soluciones de trading algorítmico existentes en el mercado actual y sus limitaciones, especialmente desde una perspectiva académica y educativa (Lusardi & Mitchell, 2014).

En el ámbito comercial, herramientas líderes como *MetaTrader* o *TradingView* dominan el sector minorista. Sin embargo, presentan barreras significativas: operan frecuentemente como “cajas negras” donde el motor de ejecución es opaco, o requieren el uso de lenguajes de programación propietarios y de nicho (como *MQL* o *PineScript*). Estos lenguajes carecen de integración nativa con el ecosistema moderno de Inteligencia Artificial (Goodfellow y col., 2016), lo que obliga a los usuarios a depender de complejas pasarelas de red para conectar sus modelos predictivos (Newman, 2015). Por otro lado, plataformas en la nube como *3Commas* o *Cryptohopper* facilitan la automatización, pero a costa de suscripciones mensuales y una nula capacidad de personalización algorítmica profunda (Hasso y col., 2019).

En el espectro del software de código abierto, existen proyectos muy consolidados. Plataformas como *QuantConnect* ofrecen una potencia institucional, pero su altísima complejidad técnica y su dependencia de infraestructuras en la nube las hacen poco accesibles para estudiantes que buscan un primer contacto con el trading cuantitativo (Lusardi & Mitchell, 2014). Por su parte, *Freqtrade* (Freqtrade Development Team, 2023) es una herramienta excelente y accesible, pero su arquitectura es monolítica (puramente en Python) (Freqtrade Team, 2026). Esto significa que la misma aplicación gestiona la red, la base de datos, la contabilidad y la IA, lo que no representa el estándar de la industria del desarrollo de software empresarial, donde estos componentes suelen estar desacoplados (Richards, 2022).

Frente a estas alternativas, este Trabajo de Fin de Grado se posiciona y diferencia al proponer una arquitectura híbrida, modular y con un enfoque netamente educativo (Chan, 2013). A diferencia de las opciones comerciales, el sistema es transparente, local y utiliza Python como lenguaje principal para la formulación de estrategias, abriendo la puerta de forma nativa al estado del arte del aprendizaje automático (López de Prado, 2018). A diferencia de las soluciones de código abierto monolíticas, este proyecto separa drásticamente las responsabilidades: un orquestador Java (VMware Tanzu & Spring Team, 2024) garantiza la robustez, concurrencia e integridad transaccional típica de los sistemas bancarios (Oracle Corporation, 2024b) (gestionando el libro mayor y la operativa simulada (Chan, 2013)), mientras que subprocesos independientes de Python actúan únicamente como el motor matemático (Hilpisch, 2018).

Esta clara separación no solo resuelve el problema técnico de la ingesta masiva de datos y su persistencia segura (Kimball & Caserta, 2004), sino que proporciona al estudiante una herramienta de experimentación inmejorable (Lusardi & Mitchell, 2014): expone una interfaz flexible capaz de ajustar los parámetros de los distintos modelos predictivos integrados, e incorpora además un módulo de búsqueda automática de la configuración óptima basado en optimización bayesiana (Feurer & Hutter, 2019), todo ello en un entorno seguro y replicable (López de Prado, 2018).

6. Análisis y Metodología

6.1. Vinculación con la Problemática Inicial

En el Capítulo 3 se establecieron tres problemas de partida que justifican el desarrollo de la plataforma: Primero, la dificultad de reacción del inversor minorista ante un mercado cripto continuo y altamente volátil. Segundo, la barrera técnica para adoptar trading algorítmico de forma segura. Tercero, el sesgo emocional en la toma de decisiones de inversión.

Este capítulo se estructura para responder de forma explícita a dichos problemas. En primer lugar, la arquitectura por capas y el orquestador Java abordan el segundo problema al desacoplar responsabilidades y reducir la complejidad operativa del sistema para su uso académico. En segundo lugar, el pipeline ETL, la persistencia masiva y la gestión de concurrencia permiten tratar el primer problema, al sostener la ingesta continua de datos y el procesamiento oportuno de velas de mercado. Por último, los módulos de backtesting, *paper trading* e IA se orientan a mitigar el tercer problema, al desplazar decisiones discrecionales hacia reglas reproducibles y validables con evidencia empírica.

6.2. Metodología de Desarrollo

Para la concepción y construcción de esta plataforma, se ha adoptado una metodología de desarrollo de software iterativa e incremental inspirada en los principios ágiles (Martin, 2003). Dada la naturaleza investigadora del Trabajo de Fin de Grado y la complejidad de integrar múltiples tecnologías (Java, Python e Inteligencia Artificial), este enfoque ha permitido una adaptación flexible ante los retos técnicos descubiertos durante el desarrollo.

El ciclo de vida del proyecto se ha estructurado en las siguientes fases metodológicas:

1. **Análisis y Toma de Requisitos:** Identificación de las necesidades del sistema, enfocadas en la creación de un entorno seguro para estudiantes y la capacidad de procesar grandes volúmenes de datos financieros.
2. **Diseño de la Arquitectura:** Definición de la arquitectura hexagonal (dominio, aplicación, infraestructura e interfaz CLI), los puertos y adaptadores, el modelo de datos y el protocolo de comunicación entre el orquestador Java y los motores Python. Los artefactos resultantes de esta fase son los diagramas estructurales y de comportamiento que se detallan en esta sección.

3. **Implementación Políglota:** Codificación de la solución dividiendo el trabajo en incrementos funcionales (primero el sistema de carteras digitales, luego la extracción de datos, y finalmente los motores de backtesting y tiempo real).
4. **Pruebas y Validación:** Ejecución de simulaciones para garantizar la transaccionalidad de la base de datos y la precisión matemática de los modelos.

6.2.1. Incrementos de Implementación

La fase de implementación se articuló en ocho incrementos funcionales, cada uno con un objetivo delimitado y criterios de cierre explícitos que aseguraban la integración correcta con el resto del sistema antes de avanzar al siguiente.

El **primer incremento** sentó las bases estructurales del proyecto. Se definió un modelo de datos relacional completo con entidades de usuario, cartera digital, estrategia, posición, libro mayor y vela, todas con borrado lógico para preservar el historial. La organización inicial del código seguía una estructura convencional de capas con controladores, servicios y repositorios Spring, sin separación estricta entre dominio e infraestructura. En paralelo, se construyó el servicio de contabilidad transaccional como único punto de acceso a operaciones monetarias, con bloqueos de escritura exclusiva a nivel de base de datos para evitar condiciones de carrera en los saldos. La interfaz de línea de comandos quedó operativa, estando algunos de los comandos protegidos por autenticación con contraseña cifrada mediante BCrypt.

El **segundo incremento** abordó la descarga incremental de datos históricos de Binance. Se implementó un algoritmo que recupera únicamente las velas posteriores a la última marca de tiempo almacenada, descargando en paralelo múltiples segmentos temporales y reparando automáticamente los huecos detectados en la serie. La persistencia se realizó mediante inserciones masivas con semántica idempotente, de manera que repetir una descarga nunca corrompe los datos existentes. Se añadió también gestión adaptativa del límite de peticiones de la API, con esperas progresivas ante respuestas de error por exceso de tráfico. El módulo fue validado con más de 50 000 registros en escenario de inserción masiva.

El **tercer incremento** implementó el cálculo vectorizado de indicadores técnicos. El motor Python calcula sobre las series de velas indicadores de tendencia, momento y volatilidad, entre ellos media móvil simple y exponencial, RSI, MACD, ATR normalizado, Bandas de Bollinger y ROC, usando operaciones matriciales de NumPy y Pandas para evitar bucles explícitos. El orquestador Java gestiona el envío del conjunto de datos al motor mediante el protocolo de comunicación entre procesos y persiste el resultado. Un período de calentamiento dinámico, calculado como el máximo de todos los períodos de los indicadores activos, descarta las primeras filas de cada serie hasta que todos los valores están estabilizados. La cobertura de pruebas del módulo de estrategias alcanzó el 90 %.

El **cuarto incremento** introdujo el motor de backtesting, que simula el comportamiento de una estrategia sobre el histórico disponible aplicando sus reglas de entrada y salida vela a vela. Para garantizar la validez de la simulación, los indicadores de cada vela se calculan únicamente con datos previos y las señales se ejecutan al precio de apertura de la vela siguiente, eliminando así cualquier filtración de información futura. El motor

calcula métricas financieras completas: tasa de acierto, factor de ganancia, caída máxima y ratio de Sharpe. Modela además costes de ejecución con una comisión del 0,1 % y un deslizamiento fijo del 0,03 % por operación. En el escenario más exigente probado, una ventana multianual de cinco años con más de 1 000 velas efectivas, el tiempo de ejecución se mantuvo por debajo de 30 segundos.

El **quinto incremento** construyó el motor de trading en tiempo real. Este módulo ejecuta estrategias de forma continua consultando precios de mercado y emitiendo señales que el orquestador Java procesa y contabiliza en la simulación de operativa. La concurrencia se gestiona con hilos virtuales de la JDK 21, lo que permite ejecutar múltiples estrategias activas simultáneamente sin bloquear hilos del sistema operativo. La comunicación entre el proceso Java y el proceso Python se estabilizó mediante un protocolo con prefijo de longitud explícito e identificador de correlación por petición, que elimina lecturas parciales y permite detectar respuestas duplicadas en los reintentos. Se añadió una cola de reintentos con espera exponencial para las señales que no pudieran procesarse en el primer intento.

El **sexto incremento** fue una refactorización arquitectónica profunda hacia el modelo hexagonal. A medida que el sistema crecía en complejidad, la estructura inicial de capas convencionales comenzó a mostrar fricciones: la lógica de dominio quedaba mezclada con detalles de infraestructura, los tests de unidad dependían de Spring, y añadir nuevos adaptadores de salida requería modificar código de negocio. La refactorización reorganizó todo el código en cuatro capas estrictas: dominio, aplicación, infraestructura e interfaz CLI. El dominio quedó libre de dependencias externas; la aplicación expuso puertos de entrada y salida como interfaces; la infraestructura implementó esos puertos con los adaptadores concretos de persistencia, IPC y seguridad. El resultado fue una base más mantenible, con tests de unidad independientes del contenedor Spring y con la posibilidad de sustituir cualquier adaptador sin tocar la lógica de negocio.

El **séptimo incremento** implementó el flujo de entrenamiento supervisado de modelos predictivos. A partir de los indicadores calculados, el motor genera automáticamente etiquetas en tres clases: compra, venta y neutral. El conjunto de datos se divide en proporción 80/20 respetando el orden cronológico, de modo que el conjunto de prueba contiene exclusivamente observaciones posteriores a las de entrenamiento. Se integró un catálogo de cuatro familias de algoritmos parametrizables desde Java: XGBoost, LightGBM, máquinas de vectores de soporte y redes neuronales densas. Para compensar el desequilibrio de clases observado en los datos, aproximadamente el 71 % de las velas resultaron neutras, las métricas de evaluación se ponderan por la frecuencia real de cada clase y los modelos que lo permiten ajustan automáticamente sus pesos internos.

El **octavo incremento** añadió el motor de optimización automática de los parámetros de cada modelo. Mediante optimización bayesiana, el motor explora el espacio de configuraciones posibles de forma guiada, descartando rápidamente las regiones de baja probabilidad de éxito y concentrando los ensayos en las zonas más prometedoras. El espacio de búsqueda, que incluye parámetros como el número de estimadores, la tasa de aprendizaje o la profundidad máxima del árbol según el tipo de modelo, es configurable desde Java. La configuración óptima encontrada queda almacenada como metadato de la instancia de estrategia para su uso posterior en predicción. Un límite de tiempo de 3 600 segundos protege al orquestador de sesiones de optimización de duración indefinida.

6.3. Tecnologías y Herramientas Utilizadas

La elección de las tecnologías empleadas responde a la necesidad de crear un sistema híbrido que combine la robustez transaccional con la agilidad en el análisis de datos (Hilpisch, 2018). A continuación, se detallan las herramientas principales y su justificación técnica:

Java 21 y Spring Boot 3: Se ha seleccionado como lenguaje de *orquestrador Java* principal debido a su fuerte tipado y su modelo de concurrencia multihilo, esencial para gestionar múltiples estrategias de trading simultáneas sin errores de memoria (Treleaven y col., 2013). El ecosistema Spring Boot 3 facilita la creación de una arquitectura modular mediante la inyección de dependencias y simplifica la persistencia de datos con Spring Data JPA (VMware Tanzu & Spring Team, 2024). La adopción de Java 21 permite el uso de hilos virtuales (Project Loom) (Goetz & Pressler, 2023), una innovación de la plataforma que proporciona millones de hilos ligeros sin la sobrecarga de los hilos del sistema operativo tradicionales.

Python 3.11+: Es el estándar de facto en la industria para la ciencia de datos y el aprendizaje automático (Goodfellow y col., 2016). Su sintaxis flexible permite iterar rápidamente en el diseño de estrategias lógicas (Hilpisch, 2018). En este proyecto, Python actúa como el motor de cálculo matemático, delegando en él las tareas de procesamiento de señales, descarga de datos de mercado y entrenamiento de modelos (McKinney, 2022).

Apache Maven: Utilizado como gestor de dependencias y herramienta de construcción para el componente Java. Permite estandarizar el ciclo de vida del proyecto, garantizando que todas las librerías se gestionen de forma centralizada y reproducible (“Apache Maven – Project Documentation”, s.f.).

MySQL 8.0+: Como sistema de gestión de bases de datos relacionales (RDBMS). Su elección se justifica por su madurez y su excelente soporte para transacciones ACID (Oracle Corporation, 2024b), lo cual es crítico para asegurar que el libro mayor y los saldos de las carteras digitales nunca queden en un estado inconsistente tras una operación técnica (Oracle Corporation, 2024a).

MessagePack: Formato binario de serialización utilizado para la comunicación interproceso (IPC) entre Java y Python. A diferencia de JSON, MessagePack ofrece un tamaño de mensaje sustancialmente menor y permite el uso de prefijado con longitud explícita, garantizando que los límites de los mensajes sean explícitos y no se corra el riesgo de lecturas incompletas o solapadas en tuberías de comunicación (Furuhashi, 2013).

6.3.1. Librerías Críticas

Para alcanzar los objetivos del proyecto, se han integrado librerías especializadas que extienden las capacidades de los lenguajes base (Richards, 2022).

Stack Java

- **Spring Data JPA:** Proporciona una capa de abstracción sobre Hibernate, permitiendo el uso del patrón Repository (Gamma y col., 1994) para consultas y persistencia sin escribir SQL manual en la mayoría de casos (Spring Team, 2024).
- **Lombok:** Utilizada para reducir el código repetitivo en las entidades y objetos de transferencia de datos, mejorando la legibilidad y el mantenimiento del código fuente mediante la generación automática de constructores por inyección de dependencias y registro estructurado de eventos (“Project Lombok: Features”, s.f.; Richards, 2022).
- **MessagePack for Jackson:** Integración entre Jackson (el serializador por defecto de Spring) y el formato MessagePack (Furuhashi, 2013), permitiendo serializar y deserializar el protocolo IPC de forma eficiente (VMware Tanzu & Spring Team, 2024).
- **Dotenv-java:** Permite la gestión segura de variables de entorno, separando la configuración sensible (credenciales de base de datos o claves de API) del código fuente (OWASP, 2024).
- **CLI interactiva:** La interfaz de línea de comandos interactiva se implementa directamente sobre las primitivas de arranque de Spring Boot 3 (VMware Tanzu & Spring Team, 2024), ejecutando un bucle de lectura de entrada estándar que parsea comandos en texto plano. Esta aproximación prescinde de *frameworks* CLI adicionales, reduciendo dependencias externas y manteniendo el control total sobre el ciclo de vida de la sesión interactiva.
- **HikariCP:** Pool de conexiones a base de datos de alto rendimiento, optimizado para aplicaciones de baja latencia (Wooldridge, 2024).

Stack Python

- **Pandas y NumPy:** Fundamentales para la manipulación eficiente de las series temporales de precios (McKinney, 2022). Permiten realizar operaciones vectorizadas sobre miles de velas en milisegundos, algo vital para la simulación histórica de estrategias (Treleaven y col., 2013).
- **Scikit-Learn, XGBoost y LightGBM:** Proporcionan las implementaciones del estado del arte para los modelos predictivos (Pedregosa y col., 2011). XGBoost, en particular, se utiliza por su capacidad de regularización para filtrar indicadores ruidosos (Chen & Guestrin, 2016). LightGBM complementa el catálogo permitiendo entrenamientos más rápidos en conjuntos de datos de gran volumen (Ke y col., 2017).
- **TensorFlow/Keras:** Plataforma de aprendizaje profundo utilizada para la construcción y entrenamiento de redes neuronales densas en los modelos de predicción de series temporales (Goodfellow y col., 2016).
- **Optuna:** Motor de optimización bayesiana de hiperparámetros (Feurer & Hutter, 2019). A partir de un espacio de búsqueda parametrizable, explora de forma guiada

las configuraciones más prometedoras y descarta automáticamente los ensayos de baja probabilidad de éxito, reduciendo el tiempo de búsqueda frente a la exploración exhaustiva. Se utiliza en el motor de optimización del proyecto para encontrar la configuración óptima de cada modelo predictivo.

- **Joblib:** Biblioteca de serialización utilizada para almacenar en disco los modelos entrenados. Permite guardar y recuperar el estado completo de un modelo con una sola llamada, lo que hace posible persistir los resultados de cada sesión de entrenamiento y reutilizarlos directamente en la predicción en tiempo real sin necesidad de reentrenar.
- **Requests:** Librería de cliente HTTP utilizada para consultar la interfaz de programación de Binance al descargar datos históricos de mercado (Binance, [2024b](#); “Requests: HTTP for Humans”, [s.f.](#)).
- **MessagePack:** Librería de serialización correspondiente a Python, permitiendo la comunicación simétrica con el lado Java (Furuhashi, [2013](#)).
- **TA:** Proporciona implementaciones de indicadores técnicos clásicos como el RSI, las medias móviles, el MACD y el ATR (Murphy, [1999](#)), integrados directamente sobre las tablas de datos de Pandas (McKinney, [2022](#)) sin dependencias nativas de compilación.

Herramientas de Calidad y Asistencia al Desarrollo

- **GitHub Copilot:** Asistente de programación basado en inteligencia artificial desarrollado por GitHub (Microsoft) (GitHub, Inc., [2024](#)). Durante el desarrollo del proyecto se ha utilizado para la generación asistida de código repetitivo, la propuesta de pruebas unitarias y la exploración de variantes de implementación, siempre bajo supervisión del desarrollador para garantizar la corrección semántica y la coherencia con la arquitectura hexagonal establecida.
- **SonarLint:** Herramienta de análisis estático integrada en el entorno de desarrollo (SonarSource S.A., [2024](#)). Proporciona retroalimentación en tiempo real sobre problemas de código, vulnerabilidades y violaciones de buenas prácticas Java directamente en el editor, adelantando la detección de problemas al momento de escritura del código antes de la fase de construcción del proyecto.
- **Checkstyle, PMD y SpotBugs:** Conjunto de herramientas de análisis estático integradas en el ciclo de construcción Maven. Checkstyle (v10.17.0) verifica la consistencia del estilo de codificación; PMD (v3.24.0) detecta fragmentos de código potencialmente problemáticos mediante análisis del árbol sintáctico abstracto; SpotBugs (v4.8.6) identifica patrones de errores conocidos mediante análisis del código compilado (OWASP, [2024](#)). Estas herramientas se configuran como barreras de calidad en la fase de verificación de Maven, impidiendo que código con violaciones supere la construcción.

- **JaCoCo:** Extensión Maven de medición de cobertura de pruebas (v0.8.12) que genera informes de cobertura por rama e instrucción tras cada ejecución de las pruebas automatizadas, permitiendo identificar áreas del sistema no ejercitadas.

6.4. Arquitectura General

La plataforma adopta como guía de diseño la arquitectura hexagonal, cuyo objetivo central es aislar la lógica de negocio de los detalles técnicos externos: la base de datos, la interfaz de usuario o la comunicación con procesos auxiliares (Martin, 2017). A diferencia de una arquitectura en capas convencional, en la que las dependencias fluyen de arriba hacia abajo, este enfoque orienta todas las dependencias hacia el núcleo del sistema, de modo que los cambios tecnológicos no obliguen a modificar las reglas del negocio (Evans, 2003).

La aplicación de este principio es pragmática y no estrictamente uniforme. El código Java se organiza en diez paquetes de primer nivel: *backtesting*, *market*, *strategy*, *trading*, *training*, *user*, *wallet*, *shared*, *config* e *interfaces*. Los siete primeros responden a capacidades funcionales del sistema; los tres últimos agrupan aspectos transversales. Dentro de los módulos más completos, como *market*, *strategy*, *trading*, *training*, *user* y *wallet*, se reconoce la separación entre dominio, aplicación e infraestructura de forma explícita. Esta organización proporciona el beneficio principal buscado: que la lógica financiera, el cálculo de beneficios y pérdidas, la gestión del libro mayor y la activación de estrategias puedan evolucionar con independencia de la base de datos, del motor de cálculo en Python o de la interfaz de usuario. La Figura 6.1 ilustra de forma esquemática esta orientación.

6.4.1. Estructura y componentes del sistema

Módulos del backend Java

La base compartida de todo el sistema reside en el paquete *shared*. La clase abstracta *BaseEntity* dota a todas las entidades de la base de datos de tres campos estandarizados: un identificador autogenerado, una marca de tiempo de creación y un indicador de borrado lógico. El uso de borrado lógico en lugar de eliminación física garantiza la trazabilidad histórica de todas las operaciones: cualquier baja se expresa como una actualización de estado sin destruir el registro original (Fowler, 2002). En un sistema contable, responder a qué capital tenía una estrategia al detenerse o por qué fue cerrada una posición es un requisito del dominio, no una conveniencia de implementación.

El módulo de **trading en tiempo real** es el núcleo operativo del sistema. La entidad *InstanciaEstrategia* modela una configuración activa de ejecución: almacena el nombre de la estrategia, el capital asignado con precisión decimal, los parámetros de riesgo y el estado de ciclo de vida, que puede ser activa, detenida o terminada. *Posicion* representa una operación de mercado individual, registrando el símbolo negociado, el precio de entrada exacto y un indicador de posición abierta que el motor consulta para evitar órdenes duplicadas sobre el mismo par. El historial financiero se construye con *LedgerEntry*, un registro inmutable de cada movimiento de capital cuyo tipo se expresa mediante *Ledger-*

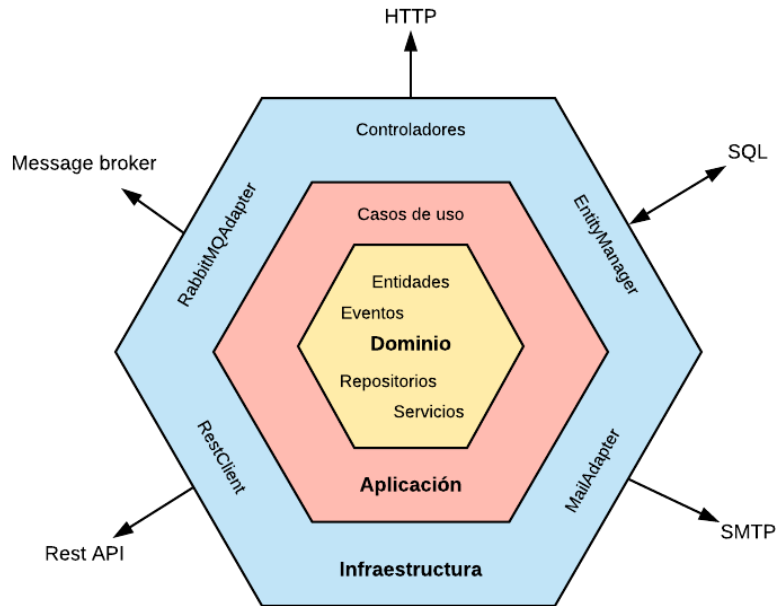


Figura 6.1: Esquema conceptual de la orientación hexagonal del proyecto; no representa literalmente toda la jerarquía de paquetes.

Type, que cubre ocho situaciones posibles: capital disponible, reservado, comprometido, margen comprometido, beneficio realizado, beneficio no realizado, comisión y estrategia pausada. La cartera digital implementa un doble saldo, formado por el balance real y el balance disponible, que impide sobregiros cuando varias estrategias compiten simultáneamente por el mismo capital. En la capa de aplicación, *PaperTradingService* simula la ejecución de órdenes y valida las reglas de riesgo, mientras que *AccountingService* actúa como único responsable de todas las operaciones monetarias, operando siempre con bloqueos de escritura para garantizar consistencia (Goetz y col., 2006). *StrategyRuntimeCoordinator* gestiona el ciclo de vida de los procesos Python de trading mediante un grupo de hilos virtuales de la JDK 21 (Goetz & Pressler, 2023), lo que permite lanzar una instancia por estrategia activa sin saturar el grupo de hilos del sistema operativo y manteniendo un modelo de programación secuencial idiomático (Goetz y col., 2006).

La robustez financiera del sistema descansa en tres garantías transaccionales complementarias. Los repositorios de cartera digital e instancia de estrategia aplican bloqueos pesimistas de escritura exclusiva, impidiendo que dos hilos modifiquen el mismo saldo de forma simultánea y evitando así las inconsistencias contables que pueden surgir en escenarios de ejecución concurrente (Goetz y col., 2006; Oracle Corporation, 2024b). Todos los movimientos monetarios pasan obligatoriamente por *AccountingService*, que los envuelve en transacciones atómicas con garantía de reversión completa ante cualquier fallo: si algún paso falla, el balance real, el balance disponible y el libro mayor vuelven al estado anterior sin dejar registros parciales (Kleppmann, 2017; Spring Team, 2024). El libro mayor actúa además como registro de auditoría permanente: dado que sus entradas son inmutables, es posible reconstruir el estado financiero de cualquier cartera en cualquier instante histórico,

lo que facilita la verificación de discrepancias (Fowler, 2002; López de Prado, 2018).

El módulo de **backtesting** permite validar el comportamiento de una estrategia sobre datos reales sin riesgo financiero (Pardo, 2008). La entidad *Vela* almacena los datos de precio y volumen con precisión decimal de ocho cifras y marca temporal en milisegundos. *VelaDTO* emplea cadenas de texto para los precios al deserializar desde la interfaz de Binance, evitando pérdidas de precisión numérica durante la transferencia. *BacktestingService* recupera las velas, serializa el mensaje en el formato binario del protocolo y lo envía al motor Python; a continuación, recibe los resultados estadísticos (tasa de acierto, factor de ganancia, caída máxima y ratio de Sharpe) y los persiste en un archivo de valores separados por comas.

El módulo de **descarga de datos de mercado** cubre el flujo de extracción, transformación y carga de datos históricos. *FetchService* implementa la sincronización incremental: consulta la última vela almacenada y solicita a Binance únicamente los datos nuevos, persistiéndolos en lotes mediante inserciones masivas nativas (Oracle Corporation, 2024d), pues la sobrecarga de seguimiento de contexto del mapeador objeto-relacional resulta inasumible a ese volumen en tiempo interactivo; la sentencia condicional de actualización garantiza además la idempotencia de las descargas repetidas y repara automáticamente huecos que pudieran haber quedado por desconexiones o cambios de interfaz, resincronizando desde el punto de ruptura sin necesidad de intervención manual (Kimball & Caserta, 2004; Pardo, 2008). El motor Python gestiona el acceso a la interfaz de Binance respetando sus límites de tasa mediante dos estrategias complementarias: de forma proactiva, introduce pausas entre peticiones según los umbrales conocidos de la interfaz; de forma reactiva, interpreta los códigos de respuesta 429 y 418 aplicando retroceso exponencial con reintentos automáticos (Binance, 2024a; Kleppmann, 2017). *MarketDataService* orquesta en una sola llamada la descarga y el cálculo de indicadores. *IndicatorsService* delega el cálculo al motor Python y persiste los resultados como entidades de indicador técnico.

El módulo de **inteligencia artificial y optimización** gestiona el entrenamiento supervisado y la búsqueda de la configuración óptima de los modelos. *AITrainingService* prepara el conjunto de datos, selecciona el tipo de modelo y lo envía al motor Python, recibiendo a cambio las métricas de evaluación. *AIOptimizationService* extiende ese flujo incorporando parámetros de búsqueda como el número de ensayos y el umbral mínimo de exactitud aceptable, delegando al motor la búsqueda bayesiana mediante Optuna (Feurer & Hutter, 2019).

El módulo de **gestión de usuarios y carteras digitales** implementa el acceso a la plataforma con las garantías de seguridad propias del software financiero (OWASP, 2024). *Usuario* es una clase Java sin dependencias de infraestructura: su persistencia se delega a un adaptador que traduce entre la representación de dominio y la entidad de base de datos mediante un mapeador dedicado. *UsuarioApplicationService* implementa los casos de uso de creación y autenticación de usuarios, aplicando cifrado de contraseña con BCrypt (Provos & Mazières, 1999): un algoritmo adaptativo que incorpora una sal aleatoria por usuario, de modo que las contraseñas nunca se almacenan en texto plano y la dificultad de cómputo puede endurecerse sin modificar el esquema de datos. Las credenciales de acceso a la base de datos se inyectan en tiempo de arranque mediante variables de entorno cargadas desde un archivo local excluido del control de versiones; si alguna variable obligatoria está

ausente, la aplicación falla de forma inmediata y visible en lugar de propagarse como un error silencioso en ejecución (Kleppmann, 2017). *WalletService* gestiona la creación y consulta de carteras en modo de operativa simulada. *SessionManager* mantiene en memoria el usuario autenticado activo, restringe el acceso a cualquier operación sensible a sesiones previamente autenticadas y registra un gancho de cierre que detiene todas las estrategias activas al apagar la aplicación (Gamma y col., 1994).

Entre las utilidades transversales, *PathConfig* centraliza la resolución de rutas del sistema de archivos con independencia del directorio desde el que se ejecute la aplicación; *AppConstants* define las constantes del contrato entre Java y Python, como claves de objetos serializados y cabeceras de archivos; y *IpcMessagePackCodec* implementa el protocolo binario de bajo nivel, encapsulado en un registro inmutable con versión de protocolo, tipo de mensaje, identificador de correlación y contenido.

Patrones de diseño aplicados

La arquitectura hace uso explícito de varios patrones de diseño reconocibles en puntos concretos del código (Fowler, 2002; Gamma y col., 1994). El patrón **Repositorio** aparece en todas las interfaces de acceso a datos, como *LedgerRepository* o *WalletRepository*, abstrayendo los detalles de consulta y persistencia frente a los servicios de aplicación. El patrón **Comando** está presente tanto en el gestor principal de la interfaz de línea de comandos como en cada clase de comando individual, que encapsula la lógica de una acción y delega su ejecución al servicio de aplicación correspondiente. El patrón **Fachada** se aplica en *PythonBridgeFacade*, que simplifica la interacción con los procesos Python detrás de una interfaz uniforme, ocultando los detalles del protocolo binario a los servicios que lo usan. El patrón **Constructor** se utiliza en *PythonBridgeRequest* para configurar cada invocación al motor Python de forma legible y sin constructores de múltiples parámetros. El patrón **Registro único** aparece en *SessionManager* gestionado por Spring, garantizando que solo exista una sesión activa en todo momento. Por último, en el motor Python, la jerarquía basada en *BaseStrategy* introduce el patrón **Estrategia**: cada implementación concreta define los mismos tres métodos obligatorios sin modificar el motor de ejecución (Gamma y col., 1994).

Motor Python

Java actúa como orquestador del sistema por su tipado estático, soporte transaccional ACID y modelo de concurrencia basado en hilos virtuales (Goetz y col., 2006); Python se incorpora como motor de cálculo por disponer del ecosistema de análisis de datos y aprendizaje automático más maduro del mercado (Goodfellow y col., 2016; Hilpisch, 2018). El subsistema Python tiene una estructura intencionalmente sencilla. Se organiza en una única carpeta de scripts, donde cada archivo corresponde a un motor que se ejecuta como un proceso independiente invocado por el orquestador Java. Los motores disponibles cubren las siguientes funciones: descarga de velas de Binance, cálculo de indicadores técnicos, simulación histórica de estrategias, trading en tiempo real con reglas clásicas, trading en tiempo real con modelo neuronal, entrenamiento supervisado y optimización de configuraciones. Todos comparten un módulo de utilidades comunes y el módulo del protocolo

de comunicación. La carpeta de estrategias, separada de los motores, contiene la clase base abstracta *BaseStrategy* y las implementaciones concretas. Esta separación garantiza que añadir una nueva estrategia no requiera modificar ningún motor existente. La descarga de datos históricos aprovecha además la concurrencia de Python mediante un grupo de hilos que lanza múltiples peticiones simultáneas a la interfaz de Binance, reduciendo significativamente el tiempo total de descarga cuando se solicitan varios periodos o símbolos a la vez (Binance, 2024b; Python Software Foundation, 2024).

6.4.2. Comunicación entre procesos

La comunicación entre el orquestador Java y los motores Python es un componente crítico de la arquitectura. Se ha definido un protocolo versionado de forma explícita para garantizar compatibilidad y permitir su evolución sin romper el funcionamiento existente (Richards, 2022).

Todo mensaje intercambiado respeta una estructura envolvente con cuatro campos: la versión del protocolo, el tipo de mensaje como código semántico de la operación (solicitud de descarga, respuesta de indicadores, error, señal en tiempo real, entre otros), un identificador de correlación único por petición para asociar solicitudes y respuestas (Hohpe & Woolf, 2003), y el contenido específico de la operación en formato binario. Para evitar problemas de sincronización en las tuberías de comunicación, cada mensaje va precedido de un entero de cuatro bytes en orden de red que especifica la longitud exacta del cuerpo. El contenido se serializa en **MessagePack** (Furuhashi, 2013), un formato binario más compacto que su equivalente en texto plano y sin pérdida de compatibilidad de tipos (Kleppmann, 2017). El volumen de datos intercambiados por sesión puede alcanzar cientos de miles de velas con sus indicadores calculados, lo que convierte la eficiencia de serialización en un requisito práctico; el prefijo de longitud de cuatro bytes simplifica además la lectura del flujo sin necesidad de delimitadores en el contenido.

La comunicación utiliza la salida estándar como canal exclusivo para el flujo de mensajes binarios, mientras que los registros y errores se envían por el canal de error estándar, garantizando que el orquestador lea el flujo de datos sin interferencias (Newman, 2015). Para el trading en tiempo real, el motor Python emite líneas de control con un prefijo de señal seguidas del contenido de la orden en formato estructurado. El coordinador Java lee cada línea y delega en el analizador de protocolo para identificar las señales; una vez reconocidas, las entrega al servicio de operativa simulada para su procesamiento contable.

La convención de códigos de salida permite al orquestador tomar decisiones de recuperación de forma automatizada (Richards, 2022): el código cero indica ejecución exitosa; el uno señala un error de lógica no recuperable sin intervención del usuario, como una estrategia inválida o un modelo no entrenado; el dos corresponde a un error en los datos de entrada; y el tres indica una interrupción por tiempo de espera agotado o cierre externo del proceso.

Para evitar que procesos bloqueados consuman recursos indefinidamente, se definen tiempos de espera diferenciados según la operación (Kleppmann, 2017): 120 segundos para el arranque del proceso, que incluye la importación de bibliotecas pesadas como TensorFlow; 300 segundos de inactividad máxima para operaciones de descarga o cálculo

de indicadores; 900 segundos para la simulación histórica, aunque el caso típico concluye en menos de un minuto (Pardo, 2008); 1800 segundos para el entrenamiento de modelos, que puede incluir validación cruzada sobre series temporales (López de Prado, 2018); y 3600 segundos para la optimización de hiperparámetros, cuya duración puede prolongarse en función del espacio de búsqueda definido. Si un proceso supera alguno de estos umbrales sin producir actividad, el orquestador lo termina forzosamente y lo registra con el código de salida tres.

6.4.3. Ventajas y desventajas de la arquitectura

La adopción de este esquema responde a una evaluación de sus beneficios frente a las necesidades del sistema (Treleaven y col., 2013).

Entre las ventajas, la más relevante es la **mantenibilidad**. Si fuera necesario cambiar el motor de base de datos de MySQL a otro sistema, solo se vería afectado el adaptador de persistencia; la lógica de las estrategias en Python y los servicios Java permanecerían intactos (Martin, 2017; Spring Team, 2024). Una consecuencia directa de esa separación es la **facilidad para escribir pruebas**: los servicios de aplicación dependen de interfaces y no de implementaciones concretas, lo que permite verificar la lógica de contabilidad sin necesidad de lanzar los procesos de inteligencia artificial ni conectarse a la base de datos real (Evans, 2003; Martin, 2017). Finalmente, la **independencia entre subsistemas** permite trabajar de forma aislada en la robustez del orquestador Java mientras se perfeccionan los modelos matemáticos en Python, siempre que el contrato de datos entre ambas partes se mantenga constante (Hilpisch, 2018; Newman, 2015).

Entre las limitaciones, la principal es el **volumen de código adicional**. El flujo de datos requiere transformar entidades en objetos de transferencia al pasar de una capa a otra, lo que incrementa el código necesario frente a una arquitectura más directa (Martin, 2017; Richards, 2022). Esta limitación se mitiga mediante el uso de Lombok para la generación automática de constructores y mapeadores dedicados (“Project Lombok: Features”, s.f.). La otra limitación es la **latencia acumulada**: en sistemas de trading de altísima frecuencia, el paso de información a través de múltiples capas puede introducir pequeños retardos (Treleaven y col., 2013). Para los intervalos de tiempo gestionados en este proyecto, velas de un minuto o superiores, este impacto es despreciable frente a la seguridad transaccional obtenida (Oracle Corporation, 2024b; Pardo, 2008).

6.5. Indicadores Técnicos Implementados

Los indicadores técnicos son herramientas matemáticas que transforman la acción del precio en señales interpretables, con el objetivo de identificar tendencias, medir impulso y detectar condiciones extremas del mercado. Murphy los clasifica en dos grandes familias (Murphy, 1999): los *seguidores de tendencia* (*trend-following*), como las medias móviles, que funcionan mejor en mercados con dirección definida; y los *osciladores*, como el RSI y el MACD, que resultan más útiles en mercados laterales o para confirmar el agotamiento de una tendencia. El proyecto implementa los representantes más canónicos de ambas familias.

La razón fundamental para incorporar indicadores técnicos en un sistema de trading algorítmico es que permiten convertir una observación subjetiva del mercado en una regla matemática objetiva y reproducible. Un analista humano podría intuir que el precio “lleva demasiado tiempo subiendo”, pero un algoritmo necesita una condición numérica precisa para actuar: $RSI > 70$, cruce del MACD, o precio por debajo de su SMA. Los indicadores son, en este sentido, el lenguaje común entre el análisis financiero y el código (Chan, 2013).

6.5.1. La Estrategia Base del Sistema

El proyecto estructura las estrategias de trading mediante una jerarquía de clases Python basada en una clase abstracta (*Abstract Base Class*) que define el contrato formal que toda estrategia debe satisfacer. La clase declara cinco métodos abstractos obligatorios: una función que recibe el conjunto de datos de velas y lo devuelve enriquecido con los indicadores calculados; dos funciones que evalúan las condiciones de apertura y cierre de posición en una vela concreta; y dos funciones que determinan los niveles de gestión del riesgo. Esta abstracción permite que los motores de *backtesting*, tiempo real, entrenamiento y optimización operen de forma genérica con cualquier estrategia registrada sin conocer sus detalles internos.

La implementación concreta de referencia tiene como propósito verificar que el pipeline completo del sistema funcione de extremo a extremo con una estrategia funcional y representativa. Define un período de calentamiento de 20 velas y calcula ocho indicadores mediante la función de cálculo de indicadores, agrupados en dos familias según su función:

Señales de trading (reglas clásicas) Evaluadas directamente por las funciones de apertura y cierre de posición: la brecha relativa entre EMA rápida y lenta, el RSI de período 6, el retorno del último *tick* y el *z-score* del volumen.

Contexto de mercado para ML Usados exclusivamente como *features* en el módulo de entrenamiento y optimización: el retorno de tres velas, el ATR normalizado, el *Percent B* de Bollinger y la tasa de cambio de 5 períodos.

Los indicadores de ambas familias se describen en las subsecciones siguientes.

6.5.2. Indicador: Media Móvil Simple (SMA)

La Media Móvil Simple es, según Murphy, el indicador técnico más ampliamente utilizado y el punto de partida para cualquier análisis de tendencia (Murphy, 1999). Su funcionamiento se basa en suavizar la acción del precio calculando el promedio aritmético de los últimos n precios de cierre, eliminando el ruido de las fluctuaciones diarias y revelando la dirección subyacente del mercado:

$$SMA_n(t) = \frac{1}{n} \sum_{i=0}^{n-1} \text{close}(t - i)$$

Por qué es una buena idea usarla. La SMA cumple una función esencial en cualquier sistema algorítmico: actúa como filtro de tendencia. Operar en la dirección de la tendencia es uno de los principios más consolidados del trading; la SMA permite cuantificarlo de forma objetiva. Cuando el precio se mantiene por encima de su media, el sistema restringe las operaciones bajistas y favorece las alcistas, evitando que el algoritmo opere “contra la corriente”. Además, su cálculo es computacionalmente trivial, lo que la hace ideal para su uso en tiempo real sobre grandes volúmenes de velas sin impacto en la latencia.

La interpretación operativa es directa: cuando el precio cotiza por encima de su media móvil, la tendencia es alcista; cuando cotiza por debajo, es bajista. Murphy enfatiza especialmente el valor de los cruces entre dos medias de distinto período como señales de entrada y salida: el cruce de la media de 50 períodos por encima de la de 200 (*Golden Cross*) marca el inicio potencial de una tendencia alcista sostenida, mientras que el cruce inverso (*Death Cross*) anticipa una tendencia bajista (Murphy, 1999).

La limitación inherente de la SMA es su *time lag*: al asignar igual peso a todas las observaciones de la ventana, reacciona con cierto retraso ante los giros del mercado. A mayor período n , mayor suavizado pero también mayor retardo en la señal.

6.5.3. Indicador: Media Móvil Exponencial (EMA)

La Media Móvil Exponencial surge como respuesta directa a la limitación de retardo de la SMA. A diferencia de ésta, la EMA asigna un peso exponencialmente decreciente a las observaciones pasadas, otorgando mayor relevancia a los precios más recientes y reaccionando con mayor agilidad ante los cambios de tendencia (Murphy, 1999):

$$\text{EMA}_n(t) = \alpha \cdot \text{close}(t) + (1 - \alpha) \cdot \text{EMA}_n(t - 1)$$

donde el factor de suavizado $\alpha = \frac{2}{n+1}$ controla la velocidad de adaptación: valores altos de α (períodos cortos) producen una EMA más reactiva pero más ruidosa, mientras que valores bajos (períodos largos) generan una curva más suave pero más lenta. La EMA aborda este problema priorizando matemáticamente los datos recientes, lo que la hace especialmente valiosa en mercados de alta volatilidad como las criptomonedas (Katsiampa, 2017).

Por qué es una buena idea usarla. En un sistema de trading en tiempo real, la velocidad de reacción ante un cambio de tendencia tiene un impacto directo en la rentabilidad: entrar tarde en una tendencia implica asumir más riesgo y capturar menos beneficio. La EMA aborda este problema priorizando matemáticamente los datos recientes, lo que la hace especialmente valiosa en mercados de alta volatilidad como las criptomonedas, donde los movimientos bruscos son frecuentes y una media lenta puede generar señales obsoletas. En el contexto del proyecto, la EMA es además el bloque constructivo del MACD, por lo que su correcta implementación es un requisito previo para el indicador de impulso más utilizado del sistema.

En la estrategia de referencia del sistema, la EMA se implementa con dos períodos deliberadamente cortos: EMA rápida (período 3) y EMA lenta (período 8). Su diferencia

relativa, denominada brecha entre medias exponenciales, se define como:

$$\text{ema_gap}(t) = \frac{\text{EMA}_3(t) - \text{EMA}_8(t)}{\text{close}(t)}$$

Un valor positivo de esta métrica indica impulso alcista a corto plazo. La condición de que dicha brecha supere 0,001 exige que este impulso supere el 0.1 % del precio, filtrando cruces espurios de escaso recorrido.

6.5.4. Indicador: Relative Strength Index (RSI)

El RSI fue desarrollado por J. Welles Wilder y presentado en su obra *New Concepts in Technical Trading Systems* (1978). Murphy lo incorpora como el oscilador de referencia para medir la *velocidad y magnitud* de los movimientos de precio, evaluando si un activo se encuentra en zona de sobrecompra o sobreventa (Murphy, 1999). A diferencia de las medias móviles, el RSI no sigue la tendencia sino que la cuestiona cuando alcanza niveles extremos.

Su cálculo se basa en la relación entre las ganancias medias y las pérdidas medias durante una ventana de n períodos:

$$\text{RSI} = 100 - \frac{100}{1 + \text{RS}}, \quad \text{donde} \quad \text{RS} = \frac{\overline{\text{ganancias}_n}}{\overline{\text{pérdidas}_n}}$$

El oscilador se mueve en el rango $[0, 100]$ y su interpretación operativa sigue los umbrales canónicos establecidos por Wilder y refrendados por Murphy: una lectura por encima de 70 indica que el activo está **sobrecomprado** (*overbought*), sugiriendo que la presión compradora puede estar agotándose y que podría producirse una corrección a la baja; una lectura por debajo de 30 señala **sobreventa** (*oversold*), indicando que la presión vendedora puede estar llegando a su límite y que podría producirse un rebote.

Por qué es una buena idea usarlo. El RSI resuelve un problema fundamental del trading algorítmico: evitar comprar en máximos y vender en mínimos. Las medias móviles por sí solas son ciegas ante el agotamiento de una tendencia; pueden señalar “tendencia alcista” incluso cuando el precio lleva semanas subiendo de forma casi vertical y una corrección es estadísticamente probable. El RSI introduce una dimensión temporal de fatiga del mercado que las medias no capturan. En la estrategia de referencia del sistema, el RSI se calcula con un período de 6 velas en lugar del estándar de 14 propuesto por Wilder, priorizando la reactividad ante cambios rápidos de impulso típicos de *timeframes* cortos en criptomonedas. Este RSI de corto plazo actúa como condición de filtro: solo se permite abrir una posición de compra si no ha superado el umbral de 55 (zona neutral-alta), reduciendo la probabilidad de entrar en un movimiento alcista ya avanzado. Adicionalmente, Murphy destaca el fenómeno de la **divergencia** como una de las señales más fiables del indicador: cuando el precio marca un nuevo máximo pero el RSI no logra superarlo, se produce una divergencia bajista que advierte de una posible pérdida de impulso y reversión de tendencia, incluso antes de que el precio lo confirme (Murphy, 1999).

6.5.5. Indicador: Moving Average Convergence Divergence (MACD)

El MACD, desarrollado por Gerald Appel, es el oscilador de impulso por excelencia y ocupa un lugar central en el análisis técnico moderno tal y como lo describe Murphy (Murphy, 1999). Combina las propiedades de seguimiento de tendencia de las medias móviles con la sensibilidad de un oscilador, ofreciendo así información tanto sobre la dirección como sobre la fuerza del movimiento.

El indicador se construye en tres componentes:

- **Línea MACD:** diferencia entre la EMA de 12 y la EMA de 26 períodos. Mide la convergencia o divergencia entre ambas medias; cuando es positiva, la EMA rápida está por encima de la lenta (impulso alcista), y cuando es negativa, ocurre lo contrario.
- **Línea de señal:** EMA de 9 períodos aplicada sobre la propia línea MACD. Actúa como suavizador y genera las señales operativas.
- **Histograma MACD:** diferencia entre la línea MACD y la señal. Su utilidad principal es anticipar los cruces antes de que se produzcan: cuando las barras del histograma se reducen, un cruce de señal es inminente.

$$\text{MACD}(t) = \text{EMA}_{12}(t) - \text{EMA}_{26}(t)$$

$$\text{Signal}(t) = \text{EMA}_9(\text{MACD})(t)$$

$$\text{Histograma}(t) = \text{MACD}(t) - \text{Signal}(t)$$

6.5.6. Indicador: *Z-score* de Volumen

El volumen de negociación es el indicador más directo de la participación del mercado en un movimiento de precio: un movimiento acompañado de volumen alto refleja mayor consenso entre participantes y es estadísticamente más significativo que el mismo movimiento con volumen escaso (Murphy, 1999).

El sistema normaliza el volumen mediante el *z-score* respecto a su distribución en las últimas 20 velas:

$$\text{vol}_z(t) = \frac{V(t) - \bar{V}_{20}(t)}{\sigma_{V_{20}}(t)}$$

Un valor del *z-score* de volumen superior a cero indica que el volumen supera su media histórica reciente. En la estrategia de referencia, la condición de apertura exige que este *z-score* supere 0,5, requiriendo que el volumen supere la media en al menos medio punto de desviación típica para confirmar la señal.

6.5.7. Indicador: ATR Normalizado

El *Average True Range* (ATR), propuesto por Wilder, es el indicador de volatilidad realizada de referencia en el análisis técnico (Murphy, 1999). Mide el rango de movimiento real de cada vela, considerando posibles huecos (*gaps*) entre sesiones:

$$\text{TR}(t) = \max(\text{high}_t - \text{low}_t, |\text{high}_t - \text{close}_{t-1}|, |\text{low}_t - \text{close}_{t-1}|)$$

$$\text{ATR}_n(t) = \frac{1}{n} \sum_{i=0}^{n-1} \text{TR}(t-i)$$

En la estrategia de referencia, el ATR de 14 períodos se normaliza dividiendo entre el precio de cierre, obteniendo una medida de volatilidad relativa comparable entre activos y niveles de precio. Este indicador no interviene en las reglas clásicas de apertura, sino que aporta contexto de volatilidad como *feature* para el módulo de ML.

6.5.8. Indicador: Bandas de Bollinger

Las Bandas de Bollinger posicionan el precio actual dentro de su distribución estadística reciente, definiendo bandas superior e inferior a k desviaciones típicas de la media móvil de n períodos (Murphy, 1999):

$$\text{Banda}_{\pm}(t) = \text{SMA}_n(t) \pm k \sigma_n(t)$$

El *Percent B* cuantifica la posición relativa del precio dentro de las bandas, normalizado en el intervalo $[0, 1]$:

$$\text{bb_pct}(t) = \frac{\text{close}(t) - \text{Banda}_-(t)}{\text{Banda}_+(t) - \text{Banda}_-(t)}$$

Un valor cercano a 0 indica que el precio se aproxima a la banda inferior (posible sobreventa), mientras que un valor cercano a 1 sugiere proximidad a la banda superior (posible sobrecompra). En la estrategia de referencia se emplean los parámetros estándar ($n = 20$, $k = 2$). El indicador se usa exclusivamente como *feature* de ML.

6.5.9. Indicador: Tasa de Cambio (ROC)

La Tasa de Cambio cuantifica el porcentaje de variación del precio respecto a n períodos anteriores, midiendo directamente la velocidad del movimiento (Murphy, 1999):

$$\text{ROC}_n(t) = \frac{\text{close}(t) - \text{close}(t-n)}{\text{close}(t-n)} \times 100$$

En la estrategia de referencia se calcula la tasa de cambio de 5 períodos, que captura el impulso acumulado en los últimos cinco *ticks*. A diferencia del retorno de un solo período, el ROC proporciona una visión del movimiento sostenido en un horizonte más amplio, lo que resulta especialmente útil para que el modelo de ML distinga entre impulsos sostenidos y fluctuaciones puntuales.

6.5.10. Relación entre Indicadores y Uso Combinado

Murphy insiste en que ningún indicador debe emplearse de forma aislada (Murphy, 1999). La práctica profesional consiste en combinar indicadores de distintas familias para obtener confirmación cruzada: una señal de compra del MACD gana fiabilidad cuando el RSI se encuentra por debajo de 50 (sin llegar a sobreventa) y el precio ha superado su SMA, ya que los tres instrumentos coinciden en señalar impulso alcista desde diferentes perspectivas matemáticas. Esta filosofía de confirmación múltiple es precisamente la base sobre la que se diseñan las estrategias implementadas en este proyecto (Murphy, 1999).

Cada indicador es, individualmente, propenso a generar falsas señales en determinadas condiciones de mercado: la SMA las genera en mercados laterales, el RSI puede permanecer en zona de sobrecompra durante tendencias alcistas prolongadas, y el MACD puede producir cruces frecuentes en períodos de baja volatilidad. La combinación de indicadores actúa como un sistema de votación: solo se ejecuta una orden cuando varios instrumentos independientes coinciden, reduciendo drásticamente la tasa de falsas señales a costa de perder algunas oportunidades. En la estrategia de referencia, la apertura de una posición de compra requiere que cuatro condiciones independientes se cumplan simultáneamente: una brecha entre medias exponenciales superior a 0,001 (tendencia alcista de corto plazo), un RSI de corto plazo inferior a 55 (impulso no agotado), un retorno positivo en el último *tick* y un *z-score* de volumen superior a 0,5 (volumen por encima de su media histórica reciente). Las condiciones de venta son simétricas. Esta exigencia de confirmación múltiple es la razón por la que la función de cálculo de indicadores de la clase base está diseñada para acoger múltiples indicadores simultáneamente (Murphy, 1999).

6.5.11. Período de Calentamiento

Todos los indicadores descritos requieren un número mínimo de velas históricas antes de poder generar valores válidos. Este período inicial, conocido como *warmup period*, produce valores indefinidos que deben descartarse antes de cualquier análisis o entrenamiento de modelos, ya que su inclusión introduciría ruido en el pipeline de ML y podría sesgar las métricas de evaluación.

El período de calentamiento del sistema se fija en el máximo entre todos los indicadores activos. En la estrategia de referencia, dicho período está fijado en 20 velas, valor que cubre la ventana de estabilización de las medias de volumen y de las Bandas de Bollinger (ambas con ventana de 20 períodos). La clase base dispone de una función que calcula automáticamente el período máximo a partir de las constantes de período definidas en cada estrategia. Las filas de calentamiento se descartan antes de cualquier análisis o entrenamiento:

Esto garantiza que todos los indicadores son matemáticamente válidos en el DataFrame que alimenta los motores de backtesting y entrenamiento de modelos (McKinney, 2022).

6.6. Módulo de Inteligencia Artificial y Validación Experimental

La capacidad predictiva del sistema descansa en un módulo de aprendizaje automático supervisado implementado en Python que integra ingeniería de características dinámica, generación de etiquetas orientada al movimiento real del mercado, validación temporal rigurosa y soporte para múltiples familias de modelos (Goodfellow y col., 2016). El módulo delega en cada estrategia la definición de sus propias características, lo que garantiza que el motor de entrenamiento puede reutilizarse sin modificación mientras las estrategias evolucionan.

6.6.1. Ingeniería de características y generación de etiquetas

Para construir el conjunto de datos de entrenamiento, el motor invoca el método abstracto *populate_indicators()* de la estrategia activa, que enriquece el marco de datos con los indicadores técnicos propios de esa estrategia: si la estrategia emplea el índice de fuerza relativa y una media móvil simple de veinte períodos, el método calcula y añade esas dos columnas; si otra estrategia solo requiere el índice de fuerza relativa, el marco de datos resultante tendrá únicamente esa característica adicional. A continuación, *get_feature_columns()* extrae todas las columnas no pertenecientes al conjunto base precio-volumen (apertura, máximo, mínimo, cierre, volumen), construyendo así la matriz de características X de forma dinámica y sin duplicación de código entre estrategias (McKinney, 2022).

La generación de etiquetas sigue un enfoque prospectivo implementado en el método *get_label()*: para cada vela en el instante t , la etiqueta se determina comparando el precio de cierre de la vela siguiente $t + 1$ con el precio de cierre actual. Si el retorno supera un umbral configurable, la etiqueta es $+1$ (expectativa de subida); si cae por debajo del umbral negativo, la etiqueta es -1 (expectativa de bajada); en caso contrario, la etiqueta es 0 (mercado lateral). El umbral por defecto del 0,2 % equilibra la sensibilidad de la señal con la frecuencia de etiquetas activas (López de Prado, 2018). Este enfoque es deliberado: si las etiquetas se derivasen directamente de las condiciones de compra o venta de la estrategia, el modelo aprendería a memorizar la lógica booleana y no a predecir el mercado, produciendo métricas artificialmente altas sin rentabilidad real (Goodfellow y col., 2016).

El desequilibrio de clases es una consecuencia esperada en datos de negociación algorítmica, donde la mayoría de los períodos no activan señales comerciales (López de Prado, 2018). Este desequilibrio se aborda en dos dimensiones. Desde el punto de vista de la evaluación, las métricas se calculan con ponderación por la frecuencia real de cada clase, evitando que las clases minoritarias sean invisibles en las estadísticas de rendimiento (Pedregosa y col., 2011). Desde el punto de vista del entrenamiento, todos los modelos ajustan automáticamente el peso de cada clase mediante *compute_class_weight("balanced")* de scikit-learn, de modo que la clase más escasa contribuye con mayor peso al gradiente de optimización; cuando el desequilibrio es muy pronunciado, pueden aplicarse adicionalmente técnicas de remuestreo sintético como SMOTE (Chawla y col., 2002). En la prácti-

ca, distribuciones muy sesgadas, como el 71 % de etiquetas neutras observado en algunas combinaciones de estrategia y marco temporal, señalan una incompatibilidad entre la estrategia y el período histórico elegido y deben motivar una revisión de los umbrales de señal o una ampliación del conjunto de datos.

6.6.2. Preparación del conjunto de datos

Los datos enriquecidos con indicadores se preparan en tres etapas antes del entrenamiento. En primer lugar, el período de calentamiento necesario para la estabilización numérica de los indicadores se determina mediante *get_warmup_period()*, que devuelve el máximo de todos los períodos configurados en la estrategia; las filas iniciales que no alcanzan ese mínimo se descartan para evitar valores incompletos que distorsionarían el aprendizaje (Murphy, 1999). En segundo lugar, se realiza una limpieza defensiva del conjunto de datos: los valores infinitos se reemplazan por valores nulos y las filas que los contienen se eliminan, registrando cuántas filas se descartaron (McKinney, 2022). En tercer lugar, el conjunto resultante se divide de forma estrictamente cronológica en proporciones 80/20: el ochenta por ciento más antiguo forma el conjunto de entrenamiento y el veinte por ciento más reciente se reserva como conjunto de evaluación final, que no participa en ningún momento del proceso de entrenamiento ni de la búsqueda de hiperparámetros para preservar la causalidad temporal (Goodfellow y col., 2016; López de Prado, 2018).

Cada modelo entrenado persiste junto con sus metadatos: nombre de la estrategia, tipo de modelo, lista de características utilizadas, distribución de etiquetas del conjunto de datos y métricas de clasificación obtenidas. Este registro permite auditar bajo qué condiciones se obtuvo cada resultado y detectar el sobreajuste al histórico descrito por López de Prado (2018). El motor genera adicionalmente un archivo de valores separados por comas con las métricas de todos los ensayos de optimización, lo que habilita un análisis posterior de la convergencia del proceso.

6.6.3. Algoritmos soportados y optimización de hiperparámetros

El módulo admite cuatro familias de modelos seleccionables desde el orquestador Java sin modificar el código de evaluación. XGBoost (Chen & Guestrin, 2016) actúa como modelo de referencia gracias a su capacidad de regularización explícita, especialmente útil para filtrar indicadores técnicos ruidosos. LightGBM (Ke y col., 2017) ofrece tiempos de entrenamiento sustancialmente menores en conjuntos de datos de gran volumen mediante su estrategia de crecimiento por hojas en lugar de por niveles. Los bosques aleatorios reducen la varianza del modelo a través del muestreo por remplazo y aportan una estimación de importancia de características que orienta la revisión de la ingeniería de características. Las redes neuronales con TensorFlow / Keras (Goodfellow y col., 2016) exploran dependencias complejas en las series de precios mediante arquitecturas configurables de capas densas con abandono regularizador y detención anticipada. La disponibilidad de varios modelos facilita la comparación controlada del rendimiento entre familias de algoritmos bajo condiciones idénticas de entrenamiento y evaluación (Feurer & Hutter, 2019).

Para la búsqueda automática de la configuración óptima, el sistema emplea Optuna (Feurer & Hutter, 2019), una librería de optimización bayesiana que usa el estimador de Parzen estructurado en árbol para descartar iterativamente las regiones del espacio de búsqueda con baja probabilidad de mejora. Un podador por mediana (*MedianPruner*, con cinco ensayos de arranque y diez pasos de calentamiento) interrumpe de forma anticipada los ensayos cuyo rendimiento intermedio queda por debajo de la mediana de los ensayos anteriores, reduciendo el coste computacional total.

La evaluación de cada ensayo sigue un esquema de partición de series temporales con cinco divisiones por defecto (*TimeSeriesSplit* de scikit-learn) (Pedregosa y col., 2011): en cada partición, el modelo se entrena sobre el subconjunto cronológicamente anterior y genera predicciones sobre el subconjunto posterior, de modo que ningún dato futuro contamina la evaluación. Sobre las predicciones de la partición más reciente —la que contiene los datos más próximos al conjunto de evaluación final— se ejecuta una simulación histórica para obtener métricas de rendimiento real: tasa de acierto, factor de ganancia y ratio de Sharpe. La puntuación compuesta que guía la búsqueda es:

$$score = 0,25 \cdot F1_{vc} + 0,40 \cdot TA + 0,25 \cdot FG_{norm} + 0,10 \cdot RS_{norm} - \max(0, brecha - 0,05) \cdot 2 \quad (6.1)$$

donde $F1_{vc}$ es el F1 ponderado medio en las particiones de validación, TA es la tasa de acierto expresada como fracción entre cero y uno, $FG_{norm} = \min(FG, 5)/5$ es el factor de ganancia normalizado al intervalo $[0, 1]$, $RS_{norm} = \max(0, \min(RS/2, 1))$ es el ratio de Sharpe normalizado, y $brecha = acc_{train} - acc_{vc}$ mide el sobreajuste. La penalización se activa cuando la brecha supera el cinco por ciento, desincentivando configuraciones que memorizan el conjunto de entrenamiento sin generalizar (Goodfellow y col., 2016). La ponderación otorga mayor peso a la tasa de acierto real (0,40) que a la calidad de clasificación estadística (0,25), reflejando que el objetivo final es la rentabilidad operativa y no la optimización de una métrica de aprendizaje aislada (López de Prado, 2018).

Una vez completados todos los ensayos, el sistema selecciona los hiperparámetros del ensayo con mayor puntuación compuesta y entrena con ellos el modelo definitivo sobre la totalidad del conjunto de entrenamiento, evaluándolo sobre el conjunto de test reservado para obtener las métricas finales de exactitud, precisión, exhaustividad y F1. Los espacios de búsqueda concretos empleados para cada familia de modelos, determinados experimentalmente durante el desarrollo del proyecto, se recogen en el Anexo D.

6.7. Diagramas de Flujo

A continuación se describen en detalle los procesos principales del sistema, trazando el recorrido de cada comando desde la entrada en la CLI hasta la respuesta final al usuario. El sistema expone veinte comandos operativos que se pueden agrupar en seis familias funcionales: gestión de usuarios, gestión de capital, obtención de datos, gestión de estrategias, inteligencia artificial y consultas informativas.

6.7.1. Gestión de Usuarios y Acceso

La Figura 6.2 muestra el flujo de entrada principal del usuario a través de la interfaz de línea de comandos (CLI), incluyendo autenticación, registro y ayuda. El *dispatcher* principal de la CLI intercepta la línea introducida y la enruta al comando correspondiente mediante el registro de comandos inicializado al arrancar la aplicación. Todos los comandos operativos verifican en primer lugar que exista una sesión activa antes de ejecutar cualquier lógica; los comandos de autenticación, en cambio, comprueban lo contrario: que el usuario no haya iniciado ya sesión.

Flujo de registro (signup). El comando solicita nombre de usuario y contraseña de forma interactiva. El caso de uso de creación de usuario comprueba, dentro de una transacción de solo lectura, que no exista otro usuario activo con ese nombre en la base de datos. Si el nombre está libre, la contraseña se transforma con BCrypt antes de que el objeto usuario sea persistido: la contraseña en claro nunca se almacena ni se transmite a la capa de persistencia. Si el nombre ya existe, el sistema lo notifica al usuario y el registro no se completa. Esta decisión de validar unicidad por nombre de usuario (en lugar de por correo electrónico) simplifica el flujo de registro y la experiencia en la CLI, donde no hay formulario web que facilite la entrada de texto largo.

Flujo de autenticación (login). El comando solicita nombre de usuario y contraseña. El caso de uso de autenticación recupera el hash almacenado para ese nombre y lo coteja con la contraseña proporcionada usando BCrypt de Spring Security Crypto. Si la verificación es positiva, el gestor de sesión registra el usuario en la variable de sesión de la aplicación, habilitando todos los comandos operativos. Si las credenciales son incorrectas, el sistema notifica el fallo sin especificar qué campo falló, evitando revelar si el nombre de usuario existe en la base de datos. Al apagar la aplicación, el gestor de sesión detecta el cierre del contexto de Spring y liquida todas las estrategias activas antes de que el proceso termine, garantizando que ningún proceso Python quede huérfano.

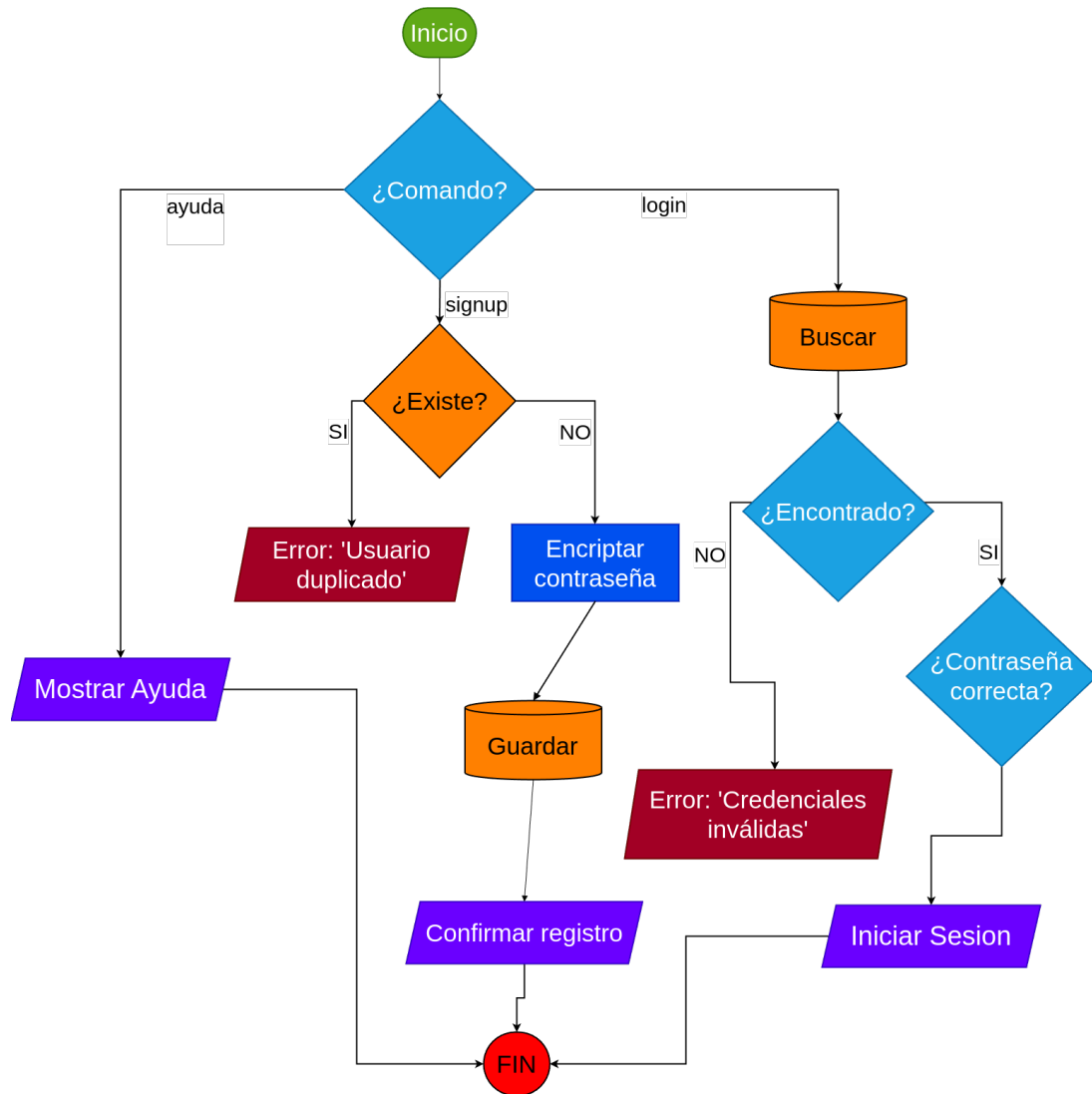


Figura 6.2: Flujo de autenticación (login/signup) y ayuda

6.7.2. Gestión de Capital (*Wallets*)

La Figura 6.3 muestra el proceso de creación y consulta de billeteras virtuales. La plataforma soporta billeteras de tipo papel (*paper trading*) para operar con capital simulado, y el ciclo de vida de cada billetera está estrechamente ligado al del libro mayor contable (*ledger*), que registra cronológicamente todos los movimientos de capital.

Listado de billeteras (1w). El comando verifica que haya sesión activa y delega en el servicio de gestión de billeteras, que consulta todas las billeteras asociadas al identificador del usuario en sesión, ordenadas por nombre. Para cada billetera se muestra el saldo real (*equity* acumulado incluido PnL realizado), el saldo disponible para nuevas operaciones y un indicador [EN USO] cuando la billetera tiene estrategias activas. Esta distinción entre saldo real y saldo disponible es fundamental: el saldo disponible se reduce en el momento en que se activa una estrategia y no se recupera hasta que ésta se liquida definitivamente, aunque se pause en medio.

Creación de billetera (mkpwallet). El comando solicita nombre y saldo inicial. El servicio de gestión de billeteras comprueba que no exista ya una billetera con ese nombre para el usuario actual. Si el nombre está disponible y el saldo es positivo, se crea la billetera con saldo real y saldo disponible iguales al depósito inicial y tipo `WalletType.PAPER`. La billetera queda en estado inactivo hasta que se asigne a una estrategia. Cuando una estrategia se activa, el servicio contable adquiere un bloqueo pesimista sobre la billetera y sobre la propia instancia de estrategia antes de restar el capital del saldo disponible; este mecanismo previene condiciones de carrera en escenarios de concurrencia en los que varias estrategias pudieran activarse simultáneamente sobre la misma billetera. En ese mismo instante se registra en el libro mayor una entrada de tipo `RESERVED` con importe negativo, garantizando la trazabilidad completa de cada movimiento de fondos desde el depósito inicial hasta la liquidación final.

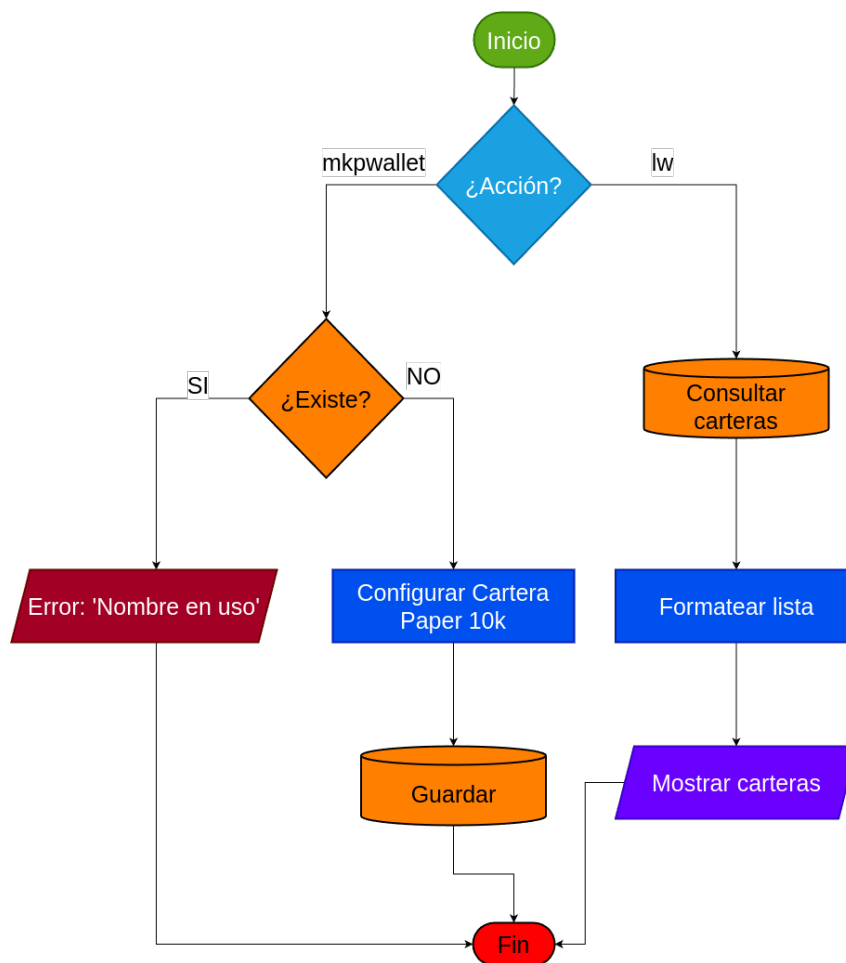


Figura 6.3: Proceso de creación y listado de *wallets*

6.7.3. Obtención de Datos (ETL)

El comando de descarga implementa un mecanismo incremental que minimiza las peticiones a la API de Binance y garantiza la consistencia del almacén histórico. El servicio de descarga distingue tres modalidades de operación según el estado de la base de datos

para el par símbolo/temporalidad solicitado.

En la modalidad por defecto, el servicio consulta la marca temporal más reciente registrada en base de datos para el par e intervalo solicitados. Si no existe ningún dato previo, lanza una descarga histórica completa desde el origen disponible en la API; si ya hay datos, calcula el primer instante ausente sumando el intervalo de temporalidad a la última marca temporal conocida y descarga únicamente las velas nuevas. En ambos casos delega en el motor Python de descarga a través del puente de comunicación: el servicio Java envía la configuración vía `MessagePack` y el motor Python responde con los datos serializados en el mismo protocolo.

El motor Python contacta el punto de acceso de velas de Binance solicitando hasta 1 500 registros por petición. Para historiales largos ejecuta hasta cinco descargas en paralelo (`MAX_PARALLEL_WORKERS = 5`) y emite las velas al consumidor Java en bloques de 2 000 registros (`EMIT_CHUNK_SIZE = 2000`) para evitar marcos IPC de tamaño excesivo. El motor aplica un tratamiento diferenciado a los errores de la API: ante un código 429 (*rate limit*) espera el número de segundos que la propia API indica en la respuesta; ante un 418, señal de bloqueo temporal de la IP, aplica una espera más larga; ante errores 5xx del servidor reintenta con *backoff* exponencial hasta un máximo de 300 segundos (`MAX_BACKOFF_SEC = 300`). Esta lógica de resiliencia permite completar descargas masivas de meses de datos sin intervención manual aunque la API imponga *throttling* intermitente.

En el lado Java, las velas recibidas se insertan en MySQL mediante operaciones por lotes con actualización condicional: si ya existe un registro con la misma marca temporal para ese par e intervalo, únicamente se actualizan el precio de cierre y el volumen, lo que hace la operación idempotente y permite reejecutar descargas sin riesgo de duplicar datos. El tamaño de lote es de 50 000 registros por operación (`BATCH_INSERT_SIZE = 50000`), lo que permite ingerir decenas de miles de velas en pocos segundos sin saturar la conexión de base de datos.

El modo de detección y relleno de huecos se activa con el argumento `--detect-gaps`. El detector itera sobre todas las marcas temporales almacenadas para el par e intervalo indicados y compara pares consecutivos. Si la diferencia entre dos marcas supera el intervalo esperado más una tolerancia de un segundo (`TOLERANCE_MS = 1 000`), se registra un hueco. La tolerancia evita falsos positivos debidos a ligeras imprecisiones de reloj en los servidores de Binance. Para cada hueco detectado, el componente realiza una descarga directa vía HTTP sobre ese rango concreto sin pasar por el proceso Python, solicitando exactamente las velas comprendidas en la ventana faltante. Este modo resulta especialmente útil cuando la base de datos ha acumulado datos de varias sesiones de descarga con interrupciones intermedias.

6.7.4. Gestión de Estrategias

La Figura 6.4 muestra el flujo de consulta y listado de estrategias. El sistema diferencia entre consultas a base de datos (instancias persistidas) y consultas al sistema de archivos (scripts Python disponibles). Los comandos `ls`, `lsa`, `lsd` y `lst` muestran, respectivamente, todas las instancias, solo las activas, solo las detenidas y solo las terminadas. En cada

caso se ejecuta una consulta filtrada por estado, excluyendo siempre los registros dados de baja lógica mediante *soft delete*. La columna de capital muestra el capital reservado para esa instancia, que puede diferir del capital comprometido en operaciones abiertas en ese instante. El comando `le` no consulta la base de datos: lee el directorio de estrategias del proyecto mediante inspección del sistema de ficheros y devuelve los módulos Python que exponen una clase válida que extiende `BaseStrategy`.

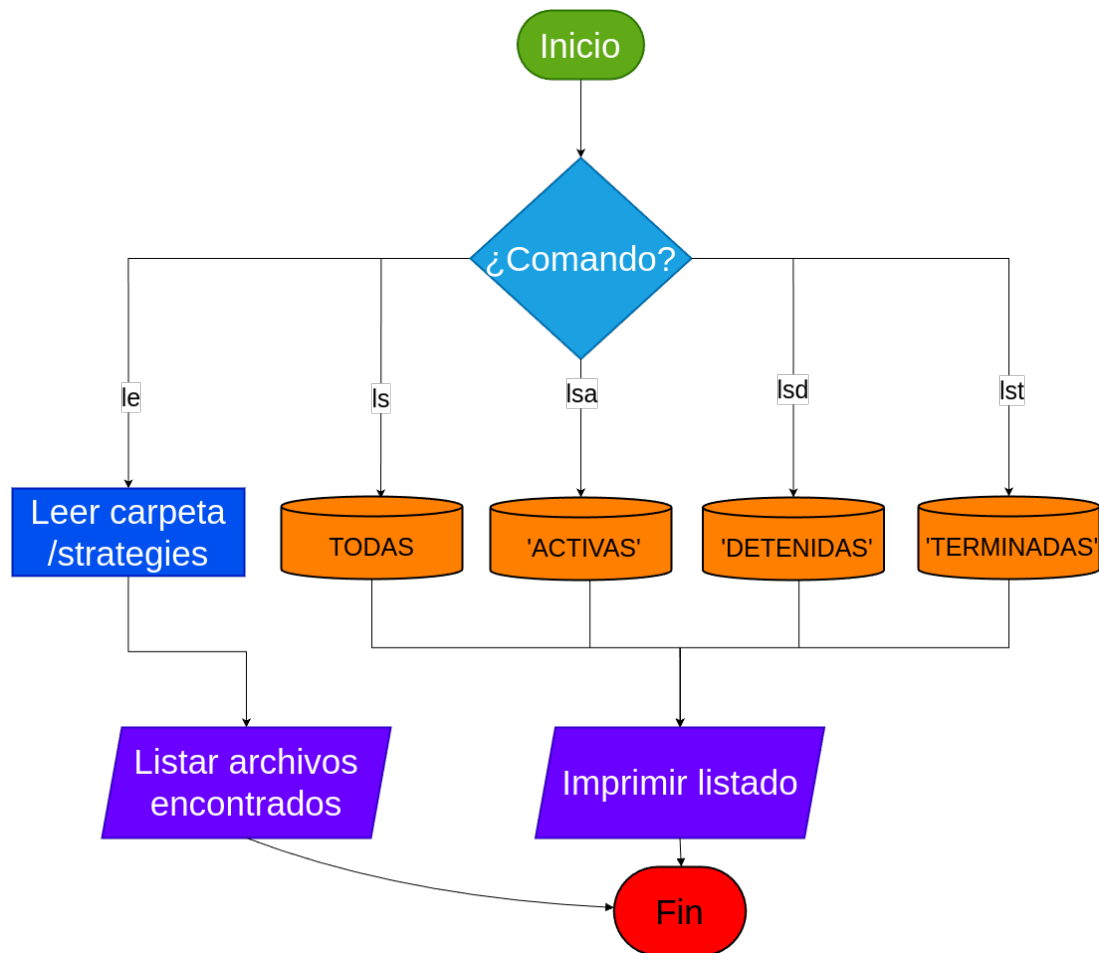


Figura 6.4: Flujo de comandos de listado (ls, le, lsa)

La Figura 6.5 detalla las operaciones de control del ciclo de vida de una estrategia: pausar (*stop*), reanudar (*start*) y liquidar (*term*).

Flujo stop. El comando `stop <id>` cancela el hilo virtual asociado a la instancia y envía la señal de interrupción al proceso Python. Acto seguido, dentro de una transacción, el servicio adquiere un bloqueo pesimista sobre la instancia de estrategia y actualiza el estado a `DETENIDA` solo si en ese momento se encontraba en estado `ACTIVA`, evitando transiciones de estado inconsistentes. Los fondos reservados para esa instancia permanecen congelados en la billetera: el saldo disponible no se recupera al pausar, lo que impide que el usuario los reasigne a otra estrategia mientras la instancia sigue persistida. La variante `stop -all` itera sobre todos los identificadores de estrategias activas y aplica el mismo flujo a cada uno.

Flujo start. El comando `start <id>` solo es aplicable a instancias en estado **DETENIDA**. El servicio de ciclo de vida recupera la instancia de la base de datos y verifica el estado antes de continuar; si el estado no es **DETENIDA**, registra un error y retorna sin modificar nada. Si el estado es correcto, actualiza el estado a **ACTIVA** dentro de una transacción y, *fuera* de ella, lanza el motor Python en un nuevo hilo virtual con los símbolos persistidos en la instancia. Este diseño, que separa deliberadamente el cambio de estado transaccional del lanzamiento del proceso Python, evita situaciones en las que el proceso quede arrancado mientras la transacción de base de datos falla y hace *rollback*.

Flujo term. El comando `term <id>` inicia la liquidación definitiva de la instancia. El servicio detiene el proceso Python de la misma forma que `stop` y, acto seguido, el componente contable adquiere un bloqueo pesimista sobre la instancia, comprueba si queda capital reservado sin comprometer y, en ese caso, lo devuelve íntegramente al saldo disponible de la billetera registrando una entrada de tipo **AVAILABLE** en el libro mayor. El estado de la instancia pasa a **TERMINADA** y ya no puede reanudarse. La variante `term -all` solicita confirmación explícita al usuario antes de proceder con todas las instancias activas.

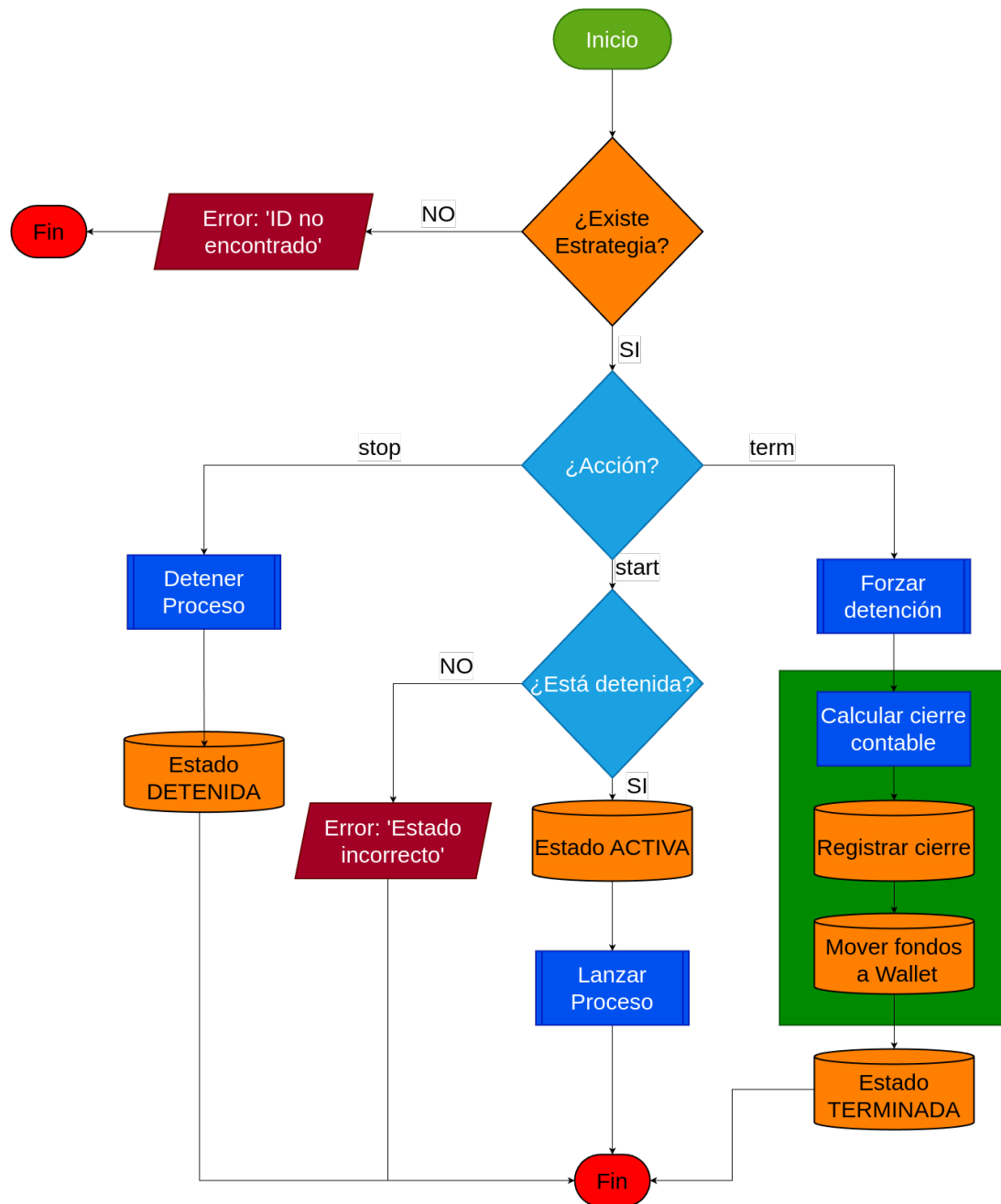


Figura 6.5: Ciclo de vida: start, stop y term

6.7.5. Backtesting

La Figura 6.6 muestra el flujo de validación histórica, disponible en modo clásico (solo estrategia de reglas) y en modo con modelo IA (estrategia más modelo supervisado).

El proceso comienza en la CLI: el comando **backtest** solicita de forma interactiva el símbolo, la temporalidad, el capital de simulación, el porcentaje de riesgo por operación, si deben borrarse los resultados previos y si los *trades* individuales deben exportarse a CSV. Con estos parámetros, el servicio de backtesting recupera de la base de datos todas las velas disponibles para esa combinación símbolo/temporalidad, ordenadas cronológi-

camente, y construye un *payload* serializado en JSON que incluye la ruta al script de estrategia, el nombre de la clase, las velas agrupadas por símbolo, el capital, el riesgo por operación y, si se especificó modelo IA, el nombre del fichero de modelo. La comunicación con Python sigue también el protocolo MessagePack con *framing* de cuatro bytes, aunque el servicio de backtesting construye el *payload* como JSON interno dentro del sobre IPC. El puente se configura sin límite de tiempo de espera dada la duración potencialmente prolongada del backtesting sobre datasets de varios años.

Dentro del entorno Python, el motor de backtesting carga la estrategia por nombre, enriquece el *DataFrame* con todos los indicadores requeridos por esa estrategia y, a continuación, itera cronológicamente sobre las velas simulando la apertura y el cierre de posiciones según las condiciones de entrada y salida definidas. Si se especificó un modelo IA, el motor lo carga desde el directorio de modelos y genera predicciones sobre el conjunto completo de velas enriquecidas; las señales de la estrategia de reglas se filtran o confirman mediante la predicción del modelo antes de ejecutarse. Al finalizar la simulación, el motor calcula las métricas de rendimiento: rentabilidad total, número de operaciones ganadoras y perdedoras, *win rate*, *drawdown* máximo y *profit factor*. Estos resultados se emiten como respuesta IPC al proceso Java.

Al recibir la respuesta, Java persiste las estadísticas del backtesting en un fichero CSV bajo el directorio **results/<estrategia>/**. Si el usuario solicitó exportar *trades*, se genera un fichero adicional con cada operación individual. El resumen de métricas se muestra al usuario en la CLI.

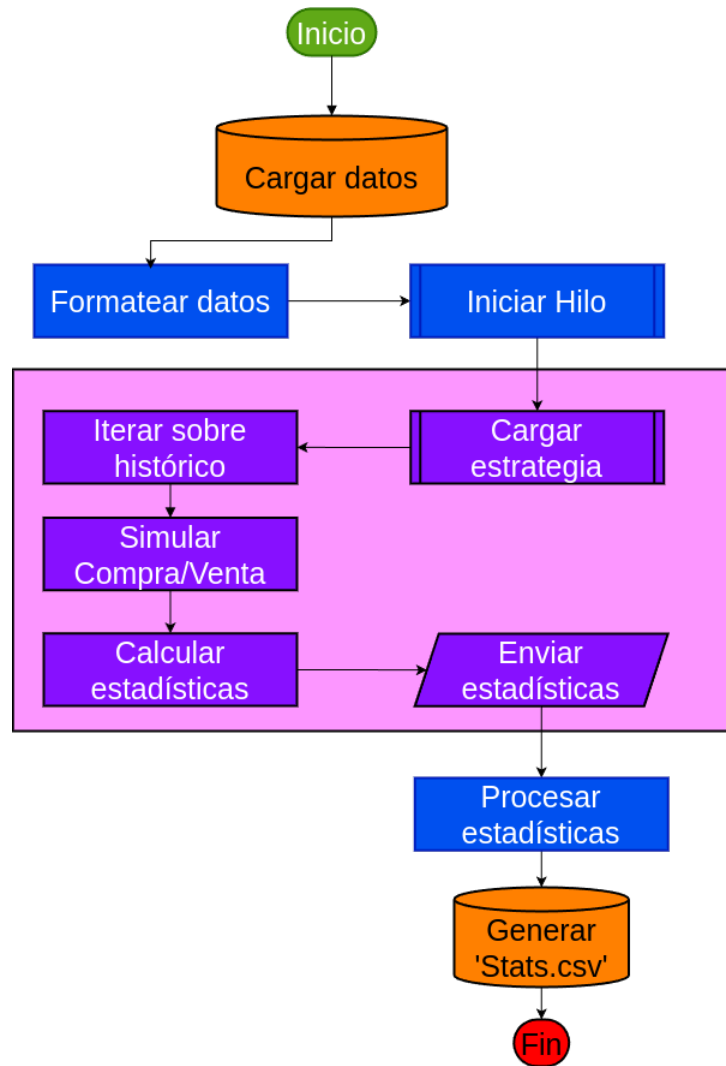


Figura 6.6: Flujo de ejecución del motor de *backtesting*

6.7.6. Trading en Tiempo Real

La Figura 6.7 muestra la arquitectura híbrida Java-Python para la operativa en tiempo real.

El proceso se inicia con el comando `trade`, que solicita la billetera de destino, el capital a asignar y el porcentaje de riesgo por operación. El servicio de ciclo de vida persiste la nueva instancia de estrategia y activa financieramente el capital dentro de una transacción, reduciendo el saldo disponible de la billetera y registrando una entrada **RESERVED** en el libro mayor. A continuación, y fuera de esa transacción para evitar situaciones en las que el proceso Python quede arrancado si el `commit` fallara, el coordinador de ejecución en tiempo real lanza el motor Python en un *Virtual Thread* de la JVM. El coordinador mantiene un registro interno que asocia cada identificador de instancia al hilo de ejecución correspondiente, permitiendo cancelar individualmente cualquier ejecución sin afectar a las restantes.

El motor Python de tiempo real recibe la configuración de la estrategia vía Mes-

sagePack y, una vez procesado el *payload*, abre una conexión WebSocket al flujo de velas de Binance para cada símbolo solicitado. Cada conexión funciona como una tarea asíncrona independiente, de modo que la plataforma puede seguir varios pares de divisas de forma concurrente dentro del mismo proceso Python. Para cada mensaje entrante, el motor extrae el precio de cierre preservando la precisión decimal completa y actualiza un búfer circular de capacidad máxima configurable (por defecto 100 velas). Una vez que el búfer supera el período de calentamiento definido por la estrategia (`WARMUP_PERIOD`), recalcula los indicadores sobre el *DataFrame* completo y evalúa las condiciones de entrada y salida sobre la última fila. Si se cumple la condición, emite una línea por la salida estándar con el prefijo `SIGNAL\t` seguido de un objeto JSON con el símbolo, la acción, el precio y el *timestamp*. Los mensajes de diagnóstico y los errores se envían por el canal de error estándar, garantizando la separación de canales. En caso de desconexión del WebSocket, el motor espera un número de segundos creciente con *backoff* exponencial (cinco segundos inicial, máximo 300 segundos) antes de intentar reconectarse.

En el lado Java, el coordinador lee la salida estándar del proceso línea a línea en el propio hilo virtual. El analizador de protocolo detecta el prefijo `SIGNAL\t` y deserializa el JSON al objeto de transferencia que representa la señal. El coordinador entrega la señal al servicio de *paper trading*, que verifica que la instancia siga en estado **ACTIVA** y discrimina la acción. Para una señal de compra, calcula el monto a invertir como el producto del capital reservado por el porcentaje de riesgo por operación y solicita al componente contable que movimiento ese margen desde la reserva general al capital comprometido en la posición, registrando simultáneamente la nueva posición como abierta. Para una señal de venta, localiza la posición abierta y calcula el PnL neto mediante la razón entre precio de salida y precio de entrada: $pnl = margin \times (exitPrice/entryPrice) - margin$; el componente contable devuelve entonces el margen más el PnL al capital reservado y actualiza el saldo real de la billetera. Las señales que no se pueden procesar de inmediato se encolan en un servicio de reintentos; si el número de fallos consecutivos supera el límite configurado, el coordinador provoca la terminación controlada de esa instancia. El coordinador también supervisa un tiempo de espera de inicialización de 90 segundos: si transcurre ese tiempo sin que el proceso Python emita ninguna línea, la instancia se considera fallida y se detiene.

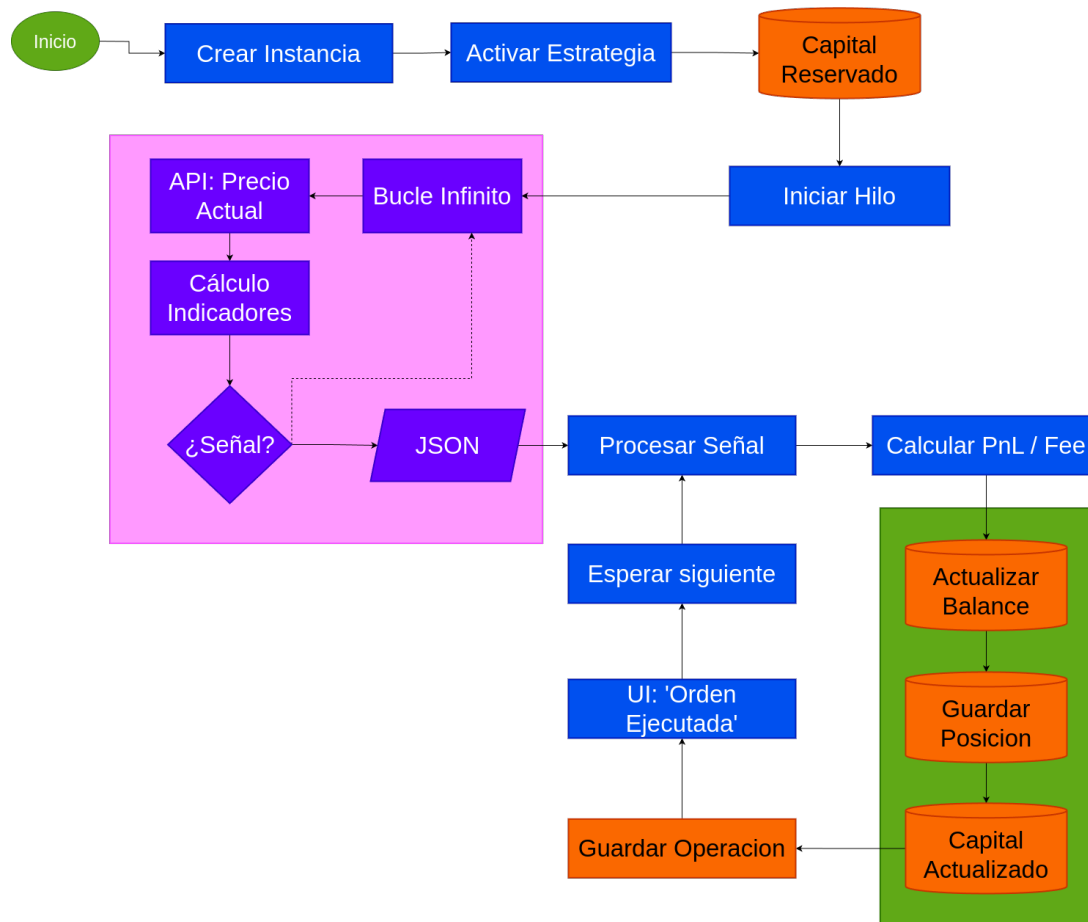


Figura 6.7: Flujo de *trading* en tiempo real y comunicación Java-Python

6.7.7. Entrenamiento de Modelos

La Figura 6.8 muestra el *pipeline* completo de aprendizaje supervisado para una combinación de modelo, estrategia, símbolo y *timeframe*.

El proceso comienza cuando el usuario invoca el comando **train** con los argumentos obligatorios: tipo de modelo, símbolo, temporalidad y nombre de estrategia. El argumento de estrategia es estrictamente obligatorio porque el sistema de entrenamiento no calcula indicadores de forma autónoma: todos los atributos del conjunto de datos se derivan de los indicadores que esa estrategia define, y prescindir de este paso produciría un modelo desacoplado de la lógica de señales que se usaría después en producción.

Antes de descargar datos, el servicio de entrenamiento determina la ventana temporal necesaria consultando al inspector de estrategias, que calcula cuántas velas históricas son necesarias para cubrir el período de entrenamiento deseado teniendo en cuenta el período de calentamiento de la estrategia. El número de velas por día depende de la temporalidad: 1 440 para minutos, 288 para cinco minutos, 96 para quince minutos, 24 para horas y así sucesivamente. Una vez determinado el número de días efectivos, el servicio lanza una descarga incremental de los datos históricos necesarios mediante el mismo ciclo ETL descrito anteriormente.

Con los datos disponibles en base de datos, el servicio construye el *payload* de entrenamiento y lo envía al motor Python de entrenamiento a través del puente de comunicación. El motor recibe la configuración vía `MessagePack`, construye el *DataFrame* y enriquece todas las filas con los indicadores de la estrategia seleccionada. A continuación extrae las características que servirán de entrada al modelo, descarta las filas del período de calentamiento y las que contengan valores no finitos, y genera las etiquetas de clase a partir de las señales de la estrategia, produciendo tres clases: +1 para compra, -1 para venta y 0 para neutro. Para mantener la integridad temporal y evitar *data leakage*, los datos se dividen en 80 % para entrenamiento y 20 % para test respetando el orden cronológico; nunca se realiza un reparto aleatorio. Las etiquetas se recodifican al rango $[0, n_clases)$ para satisfacer los requisitos internos de los modelos de XGBoost, LightGBM y redes neuronales.

Según el tipo de modelo seleccionado, el motor construye y entrena el correspondiente estimador: para los modelos clásicos (*Random Forest*, XGBoost, LightGBM, SVM) utiliza las implementaciones de la biblioteca *scikit-learn* y sus equivalentes; para redes neuronales delega en la construcción de una arquitectura profunda con Keras. En el caso de la red neuronal, y dado el desequilibrio estructural entre clases de señal habitual en mercados financieros, el motor calcula pesos de clase balanceados que ponderan el impacto de las clases minoritarias durante el entrenamiento. El modelo entrenado se serializa en el directorio `models/` bajo un nombre compuesto por tipo de modelo, temporalidad, símbolo y nombre de estrategia (con extensión `.keras` para redes neuronales y `.joblib` para modelos clásicos), acompañado de un fichero de metadatos JSON con las columnas de características y la codificación de etiquetas empleada.

El motor calcula sobre el conjunto de test las métricas de clasificación (*accuracy*, F1, precisión y *recall*) y las incluye en la respuesta IPC junto a la ruta del modelo persistido. Java recibe el sobre de respuesta y presenta el resumen de métricas en consola.

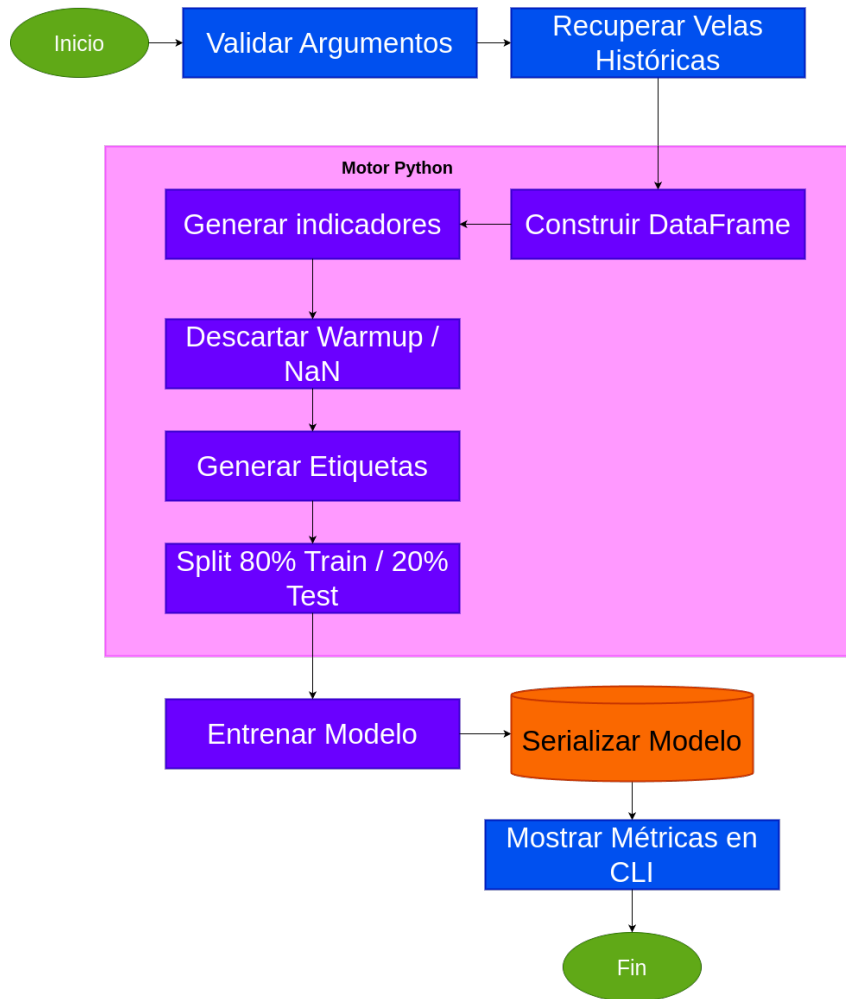


Figura 6.8: Flujo de entrenamiento de modelos supervisados

6.7.8. Optimización de Hiperparámetros

La Figura 6.9 ilustra el proceso de búsqueda automática de la combinación óptima de hiperparámetros para un modelo dado, empleando Optuna con el algoritmo *Tree-structured Parzen Estimator* (TPE). Es la operación más costosa computacionalmente del sistema: el servicio Java la invoca con un *timeout* de 3 600 segundos, y el bucle de Optuna ejecuta hasta 100 iteraciones (ensayos) por defecto, cada una de las cuales incluye validación cruzada temporal y una simulación histórica real. Los parámetros configurables desde la CLI son el tipo de modelo, el *timeframe*, el par de divisas, la estrategia Python, el umbral mínimo de *composite score* (por defecto 0,55), el número de ensayos (`-n-trials`, por defecto 100) y el número de pliegues de validación cruzada (`-cv-folds`, por defecto 5).

Preparación del dataset con separación estricta. Antes de calcular ningún indicador, el motor divide el dataset en un 80 % de entrenamiento y un 20 % de test respetando el orden cronológico. Esta decisión es intencionada: separar primero y calcular indicadores después sobre cada partición de forma independiente elimina cualquier riesgo de filtración de información futura (*data leakage*), incluso frente a indicadores con

componente global. Para que el conjunto de test disponga de las velas de calentamiento que necesita la estrategia sin solaparse con el de entrenamiento, el motor extiende la ventana de test hacia atrás `get_warmup_period()` velas (*lookback buffer*); esas filas extra se eliminan después de calcular los indicadores, de forma que el conjunto de test final comienza exactamente en el punto de corte 80/20. Tras enriquecer ambas particiones con `populate_indicators()`, se descartan las filas del período de calentamiento y las que contengan valores no finitos. Las etiquetas se generan a partir del umbral de retorno *forward-looking* de la estrategia (`LABEL_RETURN_THRESHOLD`), produciendo tres clases: compra (+1), venta (-1) y neutro (0).

Bucle de búsqueda Optuna. El estudio se crea con dirección de maximización y un podador `MedianPruner(n_startup_trials=5, n_warmup_steps=10)`: los primeros cinco ensayos nunca se podan para que el estimador de probabilidad disponga de datos suficientes; a partir del sexto, Optuna interrumpe anticipadamente cualquier ensayo cuya puntuación intermedia quede por debajo de la mediana de los ensayos completados en el mismo paso. Esto reduce significativamente el tiempo dedicado a regiones del espacio de búsqueda de baja rentabilidad.

En cada ensayo, Optuna propone una combinación de hiperparámetros según el espacio definido para el modelo seleccionado (véase el Anexo D para el detalle de rangos). El motor evalúa esa combinación mediante validación cruzada temporal con `TimeSeriesSplit`: en cada uno de los k pliegues, el conjunto de validación es siempre cronológicamente posterior al de entrenamiento, lo que respeta la causalidad de las series temporales y evita el sesgo de anticipación que introduciría un k -fold clásico aleatorizado. Para los modelos basados en *scikit-learn* (XGBoost, LightGBM, bosque aleatorio), el desbalance de clases se compensa asignando pesos inversamente proporcionales a la frecuencia de cada clase mediante `compute_class_weight("balanced")`. Para las redes neuronales, estos pesos se pasan al método `fit()` de Keras y se añade `EarlyStopping(patience=5)` sobre la pérdida de validación para evitar sobreentrenamiento dentro de cada pliegue; además, durante la fase de búsqueda, el motor limita el conjunto a las 50 000 filas más recientes para reducir el tiempo por ensayo, empleando el dataset completo únicamente en el entrenamiento final.

Una vez completados todos los pliegues de un ensayo, el motor ejecuta una simulación histórica real sobre el *fold* más reciente (el que contiene los datos cronológicamente más tardíos de entre todos los pliegues del `TimeSeriesSplit`). Esta simulación utiliza las predicciones del modelo entrenado en ese pliegue como señales de entrada y de salida para calcular las métricas de trading que no se pueden derivar de la clasificación aislada: tasa de acierto, factor de ganancia y ratio de Sharpe. La función de puntuación compuesta integra entonces todas las dimensiones de calidad del ensayo:

$$\text{score} = 0,25 \cdot F1_{cv} + 0,40 \cdot \text{WinRate} + 0,25 \cdot PF_{norm} + 0,10 \cdot \text{Sharpe}_{norm} - \text{penalización}$$

donde $PF_{norm} = \min(PF, 5,0)/5,0$ y $\text{Sharpe}_{norm} = \max(0, \min(\text{Sharpe}/2,0, 1,0))$ normalizan el factor de ganancia y el ratio de Sharpe al intervalo $[0, 1]$. La penalización se calcula como $\max(0, (\text{acc_train} - \text{acc_cv}) - 0,05) \times 2,0$: solo entra en juego cuando la brecha entre la exactitud sobre el conjunto de entrenamiento y la exactitud en validación cruzada supera el 5 %, castigando de forma progresiva los modelos que memorizan el conjunto de entrenamiento sin generalizar.

El peso de 0,40 asignado a la tasa de acierto refleja una decisión de diseño deliberada: en el contexto del trading simulado, que el modelo genere más operaciones ganadoras que perdedoras tiene un impacto directo en la rentabilidad final, más inmediato que la calidad de clasificación pura medida por el F1. Por este motivo, la tasa de acierto domina la función objetivo por encima del F1 (0,25), que penaliza los clasificadores que se especializan en una sola clase a expensas del resto.

Entrenamiento final y evaluación sobre el conjunto de test. Al terminar el bucle de Optuna, el motor identifica el ensayo con el *composite score* más alto, recupera su combinación de hiperparámetros y entrena el modelo definitivo sobre el conjunto de entrenamiento completo (el 80 % inicial). A continuación lo evalúa sobre el 20 % de test no visto durante la búsqueda: calcula exactitud, precisión, recall y F1 mediante las métricas de clasificación estándar y ejecuta una simulación histórica real con las predicciones del modelo para obtener las métricas de trading finales (tasa de acierto, factor de ganancia, caída máxima y ratio de Sharpe). El modelo se serializa con Joblib en el directorio de modelos bajo un nombre compuesto por tipo de modelo, estrategia, símbolo y *timeframe*. Si el *composite score* del mejor ensayo no alcanza el umbral mínimo configurado, el motor persiste igualmente el modelo, pero marca la respuesta con estado **warning** en lugar de **success**. Todos los ensayos completados y fallidos se guardan en un fichero CSV en el directorio de resultados bajo el subdirectorio **optimize/**, con una fila por ensayo que incluye todas las métricas intermedias y los hiperparámetros propuestos.

Respuesta al orquestador Java. Python devuelve un *envelope* IPC de tipo **OPTIMIZE_RESPONSE** que incluye: los hiperparámetros óptimos, el *composite score* máximo alcanzado y las métricas de clasificación y de trading del ensayo ganador, el *gap* de sobreajuste, la distribución de etiquetas del conjunto de entrenamiento, el número de ensayos completados, las métricas finales sobre el conjunto de test (tanto de clasificación como de simulación) y la ruta relativa del CSV con el histórico de la búsqueda. Java imprime en consola un resumen completo e informa al usuario si el umbral mínimo fue alcanzado o no.

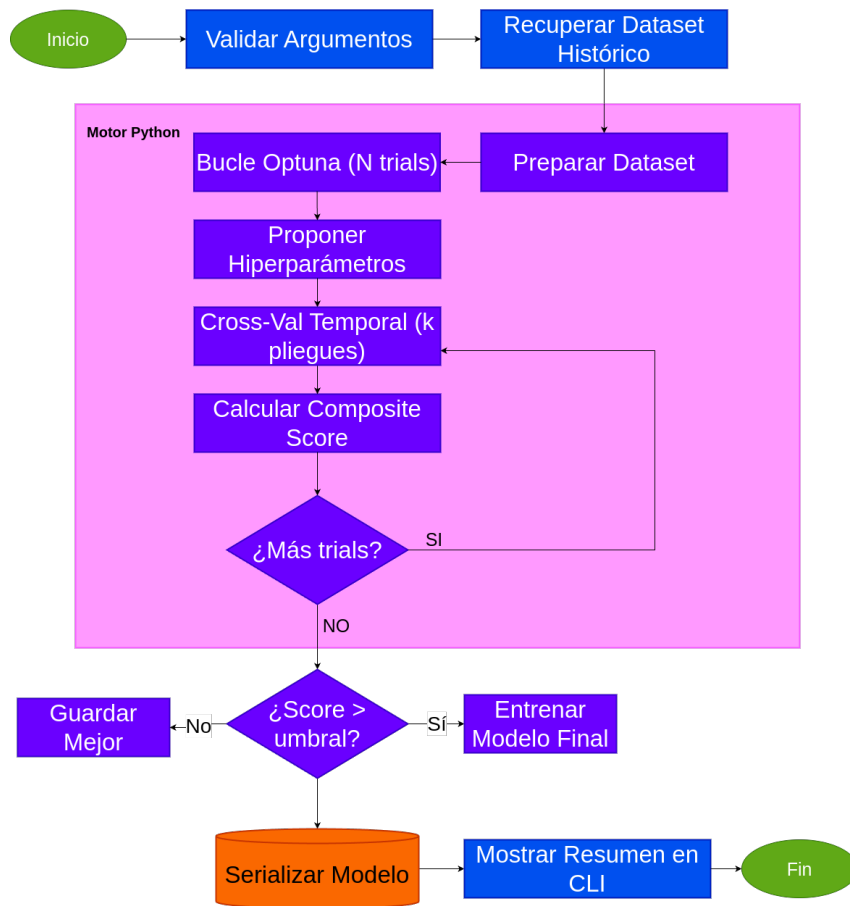


Figura 6.9: Flujo de optimización de hiperparámetros

7. Desarrollo del Trabajo

7.1. Introducción

En este capítulo se describe el desarrollo técnico y los resultados obtenidos durante la implementación del sistema de trading cuantitativo. Se detallan los incrementos funcionales completados, los benchmarks de rendimiento observados, los casos de uso validados y los problemas técnicos enfrentados y resueltos. La implementación aquí documentada materializa los objetivos del Capítulo 4, sobre la base metodológica establecida en el Capítulo 6, y sirve de base para la discusión final del Capítulo 8.

7.2. Trazabilidad de Objetivos y Evidencia

La Tabla 7.1 relaciona cada objetivo específico del Capítulo 4 con evidencia verificable del desarrollo.

Cuadro 7.1: Trazabilidad de objetivos específicos con evidencia del desarrollo

Objetivo específico	Evidencia	Métrica / valor	Sección
Estudiar el estado del arte del trading cuantitativo e indicadores técnicos	Revisión de literatura con 9 indicadores implementados (SMA, EMA, RSI, MACD, ATR, Bollinger, z-score, ROC, retorno)	9 indicadores; 21 citas bibliográficas de trading	§6.5, Cap. 5
Analizar requisitos funcionales y no funcionales de un sistema financiero de alta concurrencia	Arquitectura hexagonal con separación estricta de capas; bloqueos pesimistas en escrituras de saldo; timeouts granulares por tipo de operación	0 condiciones de carrera detectadas en tests; 5 niveles de timeout definidos	§??, §??
Diseñar una arquitectura modular que desacople orquestación transaccional del motor de inferencia	Puente Java-Python con protocolo IPC versionado; patrón Facade aislando el ciclo de vida de subprocesos; 4 capas hexagonales con dependencias unidireccionales	0 dependencias inversas entre capas verificadas en auditoría; latencia IPC <1 ms para payloads de velas	§6.4.2, §6.4.1, §6.4
Desarrollar un motor de ingesta y sincronización de datos capaz de almacenar masivamente candlesticks desde APIs externas	Pipeline ETL incremental con descarga paralela, detección de huecos y <i>batch insert</i> nativo de MySQL con semántica idempotente	>50 000 registros en inserción bulk; 60 % cobertura de tests del módulo	§??, Incremento 2
Implementar un pipeline de IA paramétrico con múltiples algoritmos y validación temporal	Motor de entrenamiento con XGBoost, LightGBM, SVM y redes neuronales; split 80/20 cronológico; warmup period dinámico; optimización bayesiana con Optuna	70 % cobertura de tests; distribución de clases 71 / 18 / 11 %	§6.6, Incrementos 6 y 7
Evaluar y validar la plataforma mediante simulaciones históricas con métricas de rendimiento financiero	Motor de backtesting con comisión 0,1 % y <i>slippage</i> 0,03 %; métricas: win rate, profit factor, max drawdown, ratio de Sharpe; backtesting multianual sobre >1 000 velas efectivas	Tiempo de ejecución típico <30s; 80 % cobertura de tests del motor	Incremento 4, Tabla 7.5

7.3. Manual de usuario

La interfaz principal de la plataforma es una línea de comandos interactiva. Para iniciar la aplicación basta con ejecutar el fichero JAR compilado desde el directorio del proyecto:

```
java -jar backendBotTrading-1.0.0.jar
```

La aplicación arranca el contexto Spring Boot, verifica la conexión con la base de datos MySQL y presenta el indicador de sesión. Mientras no hay usuario autenticado, el indicador muestra `[sin sesión]`; tras el inicio de sesión pasa a mostrar `[<nombre>]`. El comando `ayuda` lista en cualquier momento todos los comandos disponibles con su sintaxis; `exit` o `quit` cierran la aplicación de forma limpia, liberando carteras bloqueadas y deteniendo los procesos Python activos.

Gestión de cuentas y carteras. El primer paso es registrarse con `signup`, que solicita de forma interactiva nombre de usuario y contraseña; la contraseña se almacena con hash BCrypt, de modo que nunca se persiste en texto plano. Una vez registrado, `login` autentica la sesión y `logout` la cierra. La plataforma utiliza el concepto de *cartera de papel*: antes de operar es necesario crear al menos una cartera con `mkpwallet <nombre>`, que solicitará el saldo inicial en USDT que actuará como capital virtual. El comando `lw` muestra el estado de todas las carteras del usuario, desglosando el saldo disponible, el capital comprometido en operaciones abiertas y el capital bloqueado por instancias en ejecución.

Descarga de datos históricos. El comando `fetch` descarga velas OHLCV desde la API pública de Binance y las persiste en la base de datos local. Requiere especificar el marco temporal (`-tf`, por ejemplo `1h`, `4h` o `1d`) y al menos un par de divisas (`-coins`, por ejemplo `BTCUSDT`). El argumento opcional `-d <días>` acota la descarga a los últimos n días; si se omite, se descarga el histórico disponible según el marco temporal. El argumento `--detect-gaps` activa la detección y relleno automático de huecos en el histórico ya almacenado sin sobrescribir los registros existentes, lo que permite mantener la base de datos actualizada de forma incremental con ejecuciones sucesivas. Los indicadores técnicos que necesita cada estrategia (medias móviles, RSI, ATR, Bandas de Bollinger, entre otros) se calculan automáticamente en tiempo de ejecución cuando se lanza un backtesting o una sesión de trading en tiempo real, por lo que no requieren ningún paso de preprocesado adicional por parte del usuario.

Simulación histórica (*backtesting*). Antes de lanzar un backtesting conviene consultar las estrategias disponibles con `le`, que lista los scripts del directorio de estrategias junto con su descripción. El comando `backtest` ejecuta la simulación: solicita interactivamente el capital a asignar, el riesgo por operación expresado como porcentaje del capital y la comisión por operación en tanto por ciento. Si se proporciona el argumento `-model <modelo>`, el backtesting usa las predicciones del modelo de IA para filtrar las señales de

entrada, combinando la lógica de la estrategia clásica con la capa predictiva. Los resultados se exportan en ficheros CSV en el directorio de resultados e incluyen las métricas de rendimiento: tasa de acierto, factor de ganancia, caída máxima y ratio de Sharpe, junto con el detalle de cada operación abierta y cerrada durante la simulación.

Módulo de inteligencia artificial. El comando `models` muestra el catálogo de familias de algoritmos disponibles (XGBoost, LightGBM, bosque aleatorio y red neuronal) con sus hiperparámetros configurables y los valores por defecto. El comando `train` lanza el pipeline de entrenamiento supervisado: descarga y prepara el conjunto de datos, calcula los indicadores y las etiquetas de supervisión mediante la estrategia seleccionada, ejecuta el entrenamiento y persiste el modelo en disco junto con su metadata. El comando `optimize` añade una fase de búsqueda bayesiana de hiperparámetros mediante Optuna, ejecutando hasta `-n-trials` iteraciones con validación cruzada temporal de `-cv-folds` particiones; solo persiste la configuración cuando el *composite score* supera el umbral mínimo especificado con `-min-composite`.

Negociación simulada en tiempo real. El comando `trade` abre una instancia de *paper trading* que conecta con la API de Binance para recibir velas en tiempo real. El argumento `-v` emplea las señales de la estrategia clásica; `-r` usa las predicciones del modelo de IA previamente entrenado. Cada instancia tiene un identificador único que permite gestionarla de forma independiente: `lsa` lista las instancias activas con su estado y capital comprometido; `lsd` muestra las detenidas; `stop` suspende la ejecución preservando la posición abierta; `start` la reanuda; y `term` liquida la posición, calcula el resultado final y devuelve el capital no comprometido a la cartera. Las órdenes `stop -all` y `start -all` operan sobre todas las instancias a la vez.

La Tabla 7.2 recoge la referencia de comandos de gestión de cuentas, carteras, datos y backtesting. La Tabla 7.3 recoge los comandos de negociación en tiempo real e inteligencia artificial.

Cuadro 7.2: Referencia de comandos: cuenta, cartera, datos y simulación histórica

Comando	Sintaxis	Descripción
<i>Gestión de usuarios</i>		
signup	signup	Registro interactivo; contraseña almacenada con BCrypt
login	login	Autenticación; activa la sesión del usuario
logout	logout	Cierra la sesión activa
<i>Carteras virtuales</i>		
mkpwallet	mkpwallet <nombre>	Crea cartera de papel; solicita saldo inicial en USDT
lw	lw	Lista carteras con saldo disponible, comprometido y bloqueado
<i>Datos de mercado</i>		
fetch	fetch -tf <marco>-coins <par1>[par2...] [-d <días>] [--detect-gaps]	Descarga histórica de velas desde Binance; --detect-gaps rellena huecos de forma incremental
<i>Simulación histórica</i>		
le	le	Lista estrategias disponibles con descripción
backtest	backtest -strategy <nombre>-tf <marco> -coins <par1>[par2...] [-model <modelo>]	Simulación histórica; solicita capital, riesgo y comisión; -model activa filtrado por IA

Cuadro 7.3: Referencia de comandos: negociación en tiempo real e inteligencia artificial

Comando	Sintaxis	Descripción
<i>Paper trading en tiempo real</i>		
trade	<code>trade -v -r -strategy <nombre> -tf <marco>-coins <par1>[par2...]</code>	Abre instancia; -v usa señales clásicas, -r usa predicciones del modelo de IA
lsa	<code>lsa</code>	Lista instancias en ejecución
lsd	<code>lsd</code>	Lista instancias detenidas
start	<code>start <ID> start -all</code>	Reanuda una instancia o todas
stop	<code>stop <ID> stop -all</code>	Detiene una instancia o todas
term	<code>term <ID></code>	Termina la instancia, liquida la posición y libera el capital
<i>Inteligencia artificial</i>		
models	<code>models</code>	Catálogo de algoritmos con hiperparámetros y valores por defecto
train	<code>train -model <modelo>-tf <marco> -coins <par>[-strategy <nombre>] [-d <días>]</code>	Entrena y persiste el modelo con la configuración por defecto
optimize	<code>optimize -model <modelo>-tf <marco> -coins <par>[-strategy <nombre>] [-d <días>] [-n-trials <n>] [-cv-folds <k>] [-min-composite <%>]</code>	Búsqueda bayesiana de hiperparámetros con Optuna; persiste la configuración que supere el umbral mínimo

Un flujo de trabajo típico comienza con el registro mediante **signup** y el inicio de sesión con **login**. A continuación se crea una cartera de papel con **mkpwallet**, asignándole el saldo inicial en USDT que se usará como capital de operaciones. Con la cartera disponible, el siguiente paso es descargar el histórico de precios con **fetch**, indicando el marco temporal y los pares de divisas de interés; la primera descarga obtiene todo el histórico disponible, y las siguientes pueden usar **--detect-gaps** para actualizarlo de forma incremental sin duplicar registros.

Con los datos descargados, **le** muestra las estrategias disponibles y **backtest** ejecuta la primera simulación histórica. En esta fase la plataforma calcula automáticamente los indicadores técnicos que necesita la estrategia y ejecuta el backtesting sobre todos los datos del par seleccionado, exportando el detalle de operaciones y las métricas de rendimiento en CSV. Si se quiere incorporar el módulo predictivo, **train** entrena un modelo con los datos ya descargados y lo persiste en disco; **optimize** permite mejorar el resultado lanzando

una búsqueda automática de la configuración de hiperparámetros más adecuada. Una vez disponible el modelo, se puede relanzar el backtesting con `-model` para comparar el rendimiento de la estrategia clásica con el de la estrategia asistida por IA. Por último, `trade -v` o `trade -r` abre una sesión de paper trading en tiempo real que consume velas en vivo de Binance; la instancia puede pausarse, reanudarse y terminarse en cualquier momento con `stop`, `start` y `term`.

7.4. Resultados

7.4.1. Cobertura de tests automáticos

La estrategia de verificación del sistema combina suites de tests unitarios en Python y Java con una cobertura total de **466 casos de prueba** distribuidos en 20 ficheros de test. La Tabla [7.4](#) detalla la distribución por módulo.

Cuadro 7.4: Resumen de suites de tests automáticos

Fichero de test	Módulo/capa	Tests	Tecnología
test_strategies.py	Estrategias Python	48	pytest
test_backtest_engine.py	Motor de backtest-ing	41	pytest
AITrainingServiceTest	Entrenamiento IA	31	JUnit 5 + Mockito
AIOptimizationServiceTest	Optimización IA	31	JUnit 5 + Mockito
FetchServiceTest	Descarga de mercado	27	JUnit 5 + Mockito
AccountingServiceTest	Contabilidad/Ledger	24	JUnit 5 + Mockito
test_ipc_protocol.py	Protocolo IPC	23	pytest
test_engine_indicators.py	Motor de indicadores	23	pytest
StrategyLifecycleApplicationServiceTest	Servicio de test	23	JUnit 5 + Mockito
SignalRetryQueueServiceTest	Cola de reintentos	22	JUnit 5 + Mockito
BacktestingServiceTest	Servicio backtest-ing	20	JUnit 5 + Mockito
CsvUtilitiesTest	Utilidades CSV	19	JUnit 5 + Mockito
SignalProtocolParserTest	Parser de señales	16	JUnit 5 + Mockito
StrategyQueryServiceTest	Consulta de estrategias	16	JUnit 5 + Mockito
StatsCsvRepositoryTest	Repositorio de estadísticas	16	JUnit 5 + Mockito
StrategyCatalogServiceTest	Catálogo de estrategias	15	JUnit 5 + Mockito
MarketOrchestrationServiceTest	Orquestación de mercado	15	JUnit 5 + Mockito
PaperTradingServiceTest	Paper trading	14	JUnit 5 + Mockito
IndicatorsServiceTest	Servicio indicadores	12	JUnit 5 + Mockito
BacktestArtifactsCleanerTest	Limpieza de artefactos	10	JUnit 5 + Mockito
Total		466	

Los cuatro ficheros de test Python verifican la corrección aritmética y el comportamiento observable de los algoritmos cuantitativos con independencia del orquestador Java, empleando datos sintéticos de semilla fija para garantizar reproducibilidad determinista.

`test_backtest_engine.py` cubre las funciones del motor de simulación desde el cálculo de tamaño de posición hasta la escritura de operaciones en CSV; `test_engine_indicators.py` valida la adición de columnas y su serialización; `test_ipc_protocol.py` comprueba los tres modos de lectura del protocolo MessagePack con *framing* de cuatro bytes, incluyendo compatibilidad con el formato heredado; y `test_strategies.py` verifica de forma exhaustiva el contrato de `BaseStrategy` y la implementación concreta `StressTestStrategy`, desde la generación de etiquetas hasta la ausencia de valores inválidos tras el relleno defensivo.

Los dieciséis ficheros de test Java emplean JUnit 5 y Mockito con clases `@Nested` que agrupan caminos felices, condiciones de error y casos límite, sustituyendo los colaboradores externos por *mocks* para aislar la unidad bajo prueba. Las capas de mayor densidad son la de entrenamiento e inteligencia artificial (62 tests), las estrategias (54 tests), los servicios de mercado (54 tests) y el módulo de backtesting Java (45 tests). La integridad contable del *ledger* y las transiciones de estado del ciclo de vida de las instancias reciben cobertura especial dada su naturaleza transaccional; el módulo de infraestructura IPC combina 16 tests de parseo de señales y 22 de la cola de reintentos para verificar el comportamiento del puente Java-Python ante fallos y mensajes malformados.

Cuadro 7.5: Cobertura de tests por módulo funcional

Módulo	Incremento	Tests	Cobertura estimada
Contabilidad y paper trading	Inc. 1 y 5	36	82 %
Pipeline de descarga y mercado	Inc. 2	54	60 %
Motor de indicadores Python	Inc. 3	23	90 %
Estrategias Python	Inc. 3	48	90 %
Motor de backtesting Python	Inc. 4	41	80 %
Backtesting Java + artefactos	Inc. 4	45	75 %
Protocolo IPC (Python + Java)	Inc. 5	61	85 %
Servicios de estrategia (Java)	Inc. 5	54	78 %
Entrenamiento IA	Inc. 6	31	70 %
Optimización IA	Inc. 7	31	70 %
Total		424	76 %

8. Conclusiones y Vías Futuras

A. Glosario

Para la correcta comprensión del dominio del problema, se definen a continuación los conceptos económicos, financieros y técnicos fundamentales utilizados en el diseño del sistema:

Conceptos Financieros y de Trading

Exchange (Casa de Intercambio) Plataforma digital que facilita la compraventa de activos financieros, como criptomonedas, actuando como intermediario y proveedor de liquidez y datos de mercado. En este proyecto, se integra con la API REST de Binance para la obtención de datos históricos y precios en tiempo real.

OHLCV Acrónimo de *Open, High, Low, Close, Volume*. Es el formato técnico estándar para representar la acción del precio de un activo financiero durante un intervalo de tiempo específico (una vela o *candlestick*):

- **Open:** Precio al inicio del intervalo.
- **High:** Precio máximo alcanzado durante el intervalo.
- **Low:** Precio mínimo alcanzado durante el intervalo.
- **Close:** Precio al cierre del intervalo.
- **Volume:** Cantidad total del activo negociada durante el intervalo.

En el sistema, esta estructura se modela mediante la entidad JPA **Vela**.

Wallet (Billetera Digital) Entidad encargada de custodiar y gestionar el capital del usuario. En el contexto de la aplicación, se implementa un *modelo de doble saldo* para distinguir entre:

- **Balance Real:** Patrimonio total del usuario.
- **Balance Disponible:** Capital libre para nuevas operaciones (excluye fondos ya asignados a estrategias activas).

Esta separación previene sobregiros y facilita la gestión de riesgo.

Ledger (Libro Mayor) Registro contable inmutable y secuencial donde se anotan todas las transacciones financieras. Su función es garantizar la integridad de los datos y permitir la auditoría de movimientos como:

- **RESERVED:** Fondos bloqueados para una estrategia.
- **COMMITTED:** Fondos en órdenes activas de trading.
- **REALIZED_PnL:** Ganancia o pérdida realizada al cerrar una posición.
- **FEES:** Comisiones cobradas por el exchange.

En el sistema, cada movimiento genera una entrada de tipo `LedgerEntry`, permitiendo reconciliación en cualquier momento.

Estrategia de Trading Conjunto de reglas lógicas y matemáticas que determinan de forma autónoma cuándo comprar o vender un activo. Estas reglas se implementan como subclases Python que heredan de `BaseStrategy`, permitiendo:

- Definición de indicadores personalizados.
- Condiciones de entrada (`should_buy`).
- Condiciones de salida (`should_sell`).
- Cálculo de stop-loss y take-profit.

Indicador Técnico Cálculo matemático derivado del precio y/o volumen histórico de un activo, utilizado para predecir movimientos futuros del mercado. Ejemplos clave implementados:

- **RSI (Relative Strength Index):** Oscilador que mide la velocidad y el cambio de movimientos de precios (rango 0-100). RSI ≥ 30 sugiere sobreventa; RSI ≤ 70 sugiere sobrecalefacción.
- **SMA (Simple Moving Average):** Promedio aritmético de los últimos n precios de cierre.
- **EMA (Exponential Moving Average):** Media móvil que da más peso a observaciones recientes.
- **MACD (Moving Average Convergence Divergence):** Indicador de impulso basado en diferencia de dos EMAs.

El sistema persiste estos valores en la entidad `IndicadorTecnico` para análisis posterior.

Paper Trading Modalidad de simulación bursátil que permite operar con dinero ficticio en un entorno que refleja condiciones de mercado real (precios reales de Binance, pero sin dinero real comprometido). El sistema soporta este modo mediante el servicio `PaperTradingService`, permitiendo:

- Validar estrategias sin riesgo financiero.
- Educar usuarios en operativa antes de riesgo real.
- Recolectar datos de backtesting confiables.

Backtesting Proceso de validación histórica de una estrategia de trading. Consiste en simular la ejecución de una estrategia sobre datos de mercado pasados, sin estar conectado a un exchange real. Permite:

- Evaluar el rendimiento potencial antes de invertir capital real.
- Identificar estrategias débiles temprano.
- Optimizar parámetros de la estrategia.

En el sistema, el motor `engine_backtest.py` itera sobre velas históricas simulando entry/exit según la lógica de la estrategia.

Lookahead Bias Error crítico en backtesting donde el algoritmo “ve el futuro”. Por ejemplo, usar el precio de cierre de mañana para decidir si comprar hoy. Esto infla artificialmente el rendimiento. El sistema lo evita mediante *split temporal*, donde el modelo se entrena solo con datos históricos que existían en el momento de la predicción.

PnL (Profit and Loss) Término que hace referencia a las ganancias o pérdidas netas obtenidas en una operación o en un período determinado. Se calcula como:

$$\text{PnL} = \text{Precio_Salida} - \text{Precio_Entrada} - \text{Comisiones}$$

En el sistema, se registra inmediatamente en el Ledger al cerrar una posición.

Win Rate Porcentaje de operaciones rentables sobre el total de operaciones ejecutadas. Fórmula:

$$\text{Win Rate} = \frac{\text{Número de Trades Ganadores}}{\text{Número Total de Trades}} \times 100 \%$$

Un win rate de 60 % significa que 6 de cada 10 trades son ganadores (no necesariamente lucrativo si pérdidas son muy grandes).

Max Drawdown Pérdida acumulativa máxima experimentada desde un pico anterior de patrimonio. Mide el riesgo de una estrategia:

$$\text{Max Drawdown} = \frac{\text{Capital_Mínimo} - \text{Capital_Pico}}{\text{Capital_Pico}} \times 100 \%$$

Un max drawdown de -15 % significa que en el peor momento, el usuario vio su capital reducido 15 % desde su pico.

Profit Factor Relación entre ganancias totales y pérdidas totales:

$$\text{Profit Factor} = \frac{\text{Suma de PnL Positivos}}{\text{Suma Absoluta de PnL Negativos}}$$

Un profit factor >1.5 generalmente se considera aceptable.

Conceptos de Machine Learning

Feature Engineering Proceso de transformación y combinación de datos brutos en variables (*features*) que sean útiles para un modelo predictivo. En este proyecto, el **feature engineering** dinámico permite que cada estrategia defina sus propios features sin código duplicado, adaptando:

- Indicadores técnicos a calcular.
- Períodos de lookback.
- Normalizaciones o escalados especializados.

Label (Etiqueta) Variable de salida en problemas de clasificación supervisada. En este proyecto:

- **+1:** Señal de compra identificada por la estrategia.
- **-1:** Señal de venta identificada por la estrategia.
- **0:** Posición neutral (no hay señal).

Las labels se generan automáticamente mediante el método `get_label(row, next_row)` de la estrategia, permitiendo entrenamiento supervisado.

Desbalance de Clases Situación donde las clases de un dataset de clasificación no están equitativamente distribuidas. En trading, es muy común (ej., 71 % clase neutra, 18 % compra, 11 % venta; véase Capítulo 7, subsección ??). El desbalance severo causa que modelos naive predigan solo la clase mayoritaria. El sistema lo aborda con:

- Métricas ponderadas (`average='weighted'`).
- `class_weight='balanced'` en XGBoost y Random Forest.
- Documentación explícita de distribución en metadata.

Split Temporal División train/test que respeta la causalidad temporal. A diferencia de split aleatorio, en series temporales es crítico entrenar solo con datos históricos y testear en datos más recientes:

$$\text{Train: } t_1 \dots t_{70\%} \quad \text{Test: } t_{70\%+1} \dots t_{100\%}$$

Esto evita lookahead bias y valida la generalización real.

Warmup Period Número de observaciones iniciales descartadas antes de comenzar a usar indicadores técnicos o labels. Esto es necesario porque indicadores como RSI requieren al menos n períodos históricos para poder calcularse. El sistema automáticamente:

- Identifica el máximo período de todos los indicadores (ej., 26 para MACD).
- Descarta las primeras 26 velas del DataFrame.
- Asegura que todas las features sean válidas (no NaN).

Overfitting Problema donde un modelo memoriza detalles específicos del dataset de entrenamiento en lugar de aprender patrones generalizables. En trading, es especialmente peligroso porque:

- Datos históricos pueden contener patrones espurios que no se repiten.
- Un modelo overfitted parece excelente en backtesting pero falla en operativa real.

El proyecto lo mitiga con split temporal, dropout en redes neurales, y validación en conjunto de test no visto.

Accuracy Proporción de predicciones correctas sobre el total:

$$\text{Accuracy} = \frac{\text{Predicciones Correctas}}{\text{Total de Predicciones}}$$

Por sí sola, es engañosa en desbalance severo (un clasificador que siempre predice la clase mayoritaria puede tener 70 % accuracy).

F1 Score (Ponderado) Media armónica entre Precision y Recall:

$$F1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

La versión ponderada (`average='weighted'`) considera la distribución de clases, siendo más realista en datasets desbalanceados.

Matriz de Confusión Tabla que desglosa predicciones correctas e incorrectas por clase. Para un problema binario:

- **TP (True Positive):** Predicción positiva correcta.
- **FP (False Positive):** Predicción positiva incorrecta (falsa alarma).
- **TN (True Negative):** Predicción negativa correcta.
- **FN (False Negative):** Predicción negativa incorrecta (falta).

Hyperparámetros Parámetros de configuración del algoritmo de ML que no se aprenden durante el entrenamiento, sino que se especifican antes. Ejemplos:

- **XGBoost:** `n_estimators=150`, `learning_rate=0.05`, `max_depth=6`.
- **Red Neuronal:** `epochs=50`, `batch_size=64`, `learning_rate=0.001`, `dropout_rate=0.3`.

El sistema permite inyectar hiperparámetros desde Java hacia Python para experimentación sin cambiar código.

Dropout Técnica de regularización en redes neuronales donde se desactivan aleatoriamente neuronas durante el entrenamiento, reduciendo dependencias co-adaptadas. Típicamente, `dropout_rate = 0,3` (desactiva 30 % de neuronas en cada paso).

Metadata Información sobre el entrenamiento de un modelo, como:

- Strategy utilizada, símbolo, timeframe.
- Rango temporal del dataset.
- Número de features y sus nombres.
- Métricas de validación (`accuracy`, `F1`, `confusion_matrix`).
- Hyperparámetros utilizados.

Esto permite reproducir exactamente qué configuración produjo qué resultado.

Conceptos de Arquitectura e Inter-Process Communication

IPC (Inter-Process Communication) Mecanismo para intercambiar datos entre procesos distintos corriendo en el mismo o diferente máquina. En este proyecto:

- Proceso Java orquestador (PID principal).
- Procesos Python hijo (`engine_fetch.py`, `engine_train.py`, `engine_rt.py`).
- Comunicación vía `stdin/stdout` con framing `MessagePack`.

Protocolo IPC v1.0 Contrato explícito que define la estructura de todos los mensajes intercambiados:

```
{
  "protocol_version": "1.0",
  "message_type": "FETCH_REQUEST|TRAIN_RESPONSE|...",
  "correlation_id": "uuid-v4",
  "payload": { ... }
}
```

Permite evolución futura sin romper compatibilidad.

Framing Length-Prefixed Técnica de serialización donde cada mensaje está precedido de su longitud en bytes (típicamente 4 bytes big-endian):

[4 bytes length] + [body]

Esto evita problemas de sincronización en tuberías: el receptor sabe exactamente cuántos bytes esperar.

MessagePack Formato binario de serialización de datos más compacto que JSON (30-50% menor tamaño). Soporta tipos de datos complejos (arrays, maps) sin la verbosidad de JSON. En el proyecto se usa con framing length-prefixed para máxima eficiencia.

Fallback Legacy Capacidad de un protocolo nuevo de aceptar mensajes en formato antiguo para compatibilidad hacia atrás. En `ipc_protocol.py`, si se recibe un mensaje que no es `MessagePack v1`, se intenta parsear como JSON plano, permitiendo transiciones graduales.

Stdout/Stderr Separation En comunicación inter-proceso, es crítico que:

- Los datos estructurados salgan por **stdout** en binario (IPC).
- Los logs, debug y errores salgan por **stderr** en texto.

Esto previene que un `logging.info()` accidental corrompa el IPC.

Virtual Threads Innovación de Java 21 (Project Loom) que permite crear millones de hilos ligeros sin el overhead de hilos del sistema operativo. Ventajas:

- Cada estrategia ejecuta en un hilo virtual separado, sin blocking mutuo.
- Código escrito como si fueran threads tradicionales.
- Escalabilidad a decenas de estrategias simultáneas.

Se crean vía `Executors.newVirtualThreadPerTaskExecutor()`.

ExecutorService Pool de hilos en Java que ejecuta tareas de forma asíncrona. En el proyecto:

- **StrategyRuntimeCoordinator**: ExecutorService basado en Virtual Threads para lanzar y supervisar estrategias de tiempo real.
- **MarketDataService**: ExecutorService para paralelizar descarga de velas.

Transaccionalidad ACID Garantía de bases de datos donde:

- **A (Atomicity)**: Transacción se ejecuta completamente o no al todo.
- **C (Consistency)**: BD pasa de un estado válido a otro válido.
- **I (Isolation)**: Transacciones concurrentes no interfieren.
- **D (Durability)**: Cambios persistidos no se pierden tras fallo.

En el proyecto, MySQL con InnoDB garantiza ACID, permitiendo que AccountingService sea seguro bajo concurrencia.

Lock Pesimista Mecánica de base de datos que adquiere un lock exclusivo sobre una fila antes de modificarla, bloqueando otras transacciones. En `WalletRepository`:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
Optional<Wallet> findByIdWithLock(Long id);
```

Garantiza que ninguna otra transacción lee o modifica la Wallet mientras se procesa.

Soft Delete (Borrado Lógico) Patrón donde no se elimina un registro de la BD, sino que se marca como eliminado (columna `eliminado = 1`). Ventajas:

- Integridad referencial preservada.
- Auditoría conserva historial.
- Recuperación de datos accidentales es posible.

Pattern Repository Patrón de diseño que abstrae el acceso a datos. En el proyecto:

- `WalletRepository` abstrae consultas a tablas Wallet.
- `VelaRepository` abstrae consultas a tablas Vela.
- Spring Data JPA proporciona implementación automática.

Permite cambiar la base de datos sin tocar la lógica de negocio.

Facade Pattern Patrón que proporciona una interfaz simplificada a un subsistema complejo. En el proyecto:

- `MarketDataService` es una Facade que coordina `FetchService` e `IndicatorsService`.
- `PythonBridgeFacade` ordena la orquestación de procesos Python.

ETL (Extract, Transform, Load) Proceso de:

- **Extract:** Obtener datos crudos (API de Binance).
- **Transform:** Limpiar, validar, enriquecer (indicadores técnicos).
- **Load:** Persistir en BD.

El `FetchService` implementa ETL incremental optimizado.

Batch Insert Inserción masiva de múltiples filas en una sola operación. En MySQL:

```
INSERT INTO vela (...) VALUES (...), (...), (...)  
ON DUPLICATE KEY UPDATE ...
```

Es 10-100x más rápido que inserts individuales.

Rate Limiting Restricción de cuántas peticiones puede hacer un cliente a un servidor por unidad de tiempo. Binance limita a ≈ 1200 peticiones por minuto. El sistema implementa:

- Esperas proactivas entre peticiones.
- Detección reactiva de respuestas 429 (Too Many Requests).
- Respeto al header `Retry-After` cuando está presente.

Timeout Límite de tiempo para que una operación se complete. En el proyecto:

- `STARTUP_TIMEOUT_SECONDS = 120`: Máximo para que Python inicie.
- `BATCH_INACTIVITY_TIMEOUT_SECONDS = 300`: Máximo sin actividad.
- `BACKTEST_INACTIVITY_TIMEOUT_SECONDS = 900`: Para backtesting largo.

Si vence, el proceso se destruye forzosamente.

Idempotencia Propiedad de una operación donde ejecutarla múltiples veces produce el mismo resultado que ejecutarla una sola vez. En el proyecto:

- `ON DUPLICATE KEY UPDATE` en inserciones.
- `correlation_id` en envelope IPC.

Permite reintentos seguros sin duplicados.

Conceptos de Criptomonedas y Mercados

Symbol (Símbolo de Trading) Identificador de un par de monedas, ej. BTCUSDT:

- **BTC:** Bitcoin (activo base).
- **USDT:** Tether en USD (activo cotizante).

Significa “precio de 1 Bitcoin en Tether”.

Timeframe (Marco Temporal) Período de tiempo que representa cada vela (en formato OHLCV). Ejemplos:

- **1m:** 1 minuto (para scalping).
- **5m, 15m, 1h:** Comunes en swing trading.
- **1d:** 1 día (para inversión largo plazo).

El sistema soporta cualquier timeframe que Binance ofrezca.

Liquidez Capacidad de comprar o vender una cantidad significativa de un activo sin cambiar drásticamente su precio. Mayor liquidez = más operaciones posibles sin slippage (resbalamiento).

Slippage (Resbalamiento) Diferencia entre el precio esperado y el precio real al ejecutar una orden. En mercados ilíquidos, es significativo. El modelo de backtesting del proyecto asume 0.03 % de slippage fixed.

Comisión (Fee) Costo cobrado por el exchange por cada operación. Binance cobra típicamente 0.1 % del volumen. El sistema lo incluye en el cálculo de PnL:

$$\text{PnL Neto} = \text{PnL Bruto} - \text{Comisiones}$$

Conceptos de Criptografía y Seguridad

Sal (Salt) Dato aleatorio único que se añade a una contraseña antes de calcular su hash criptográfico. Cumple tres funciones clave: primero, evita que dos usuarios con la misma contraseña generen hashes idénticos (los atacantes no pueden identificar contraseñas duplicadas); segundo, invalida tablas de arcoíris (rainbow tables) precalculadas, pues cada usuario requeriría una tabla distinta; tercero, en algoritmos como BCrypt se almacena dentro del hash, permitiendo verificar la contraseña sin guardarlo en texto plano. En el sistema, BCrypt genera automáticamente una sal aleatoria por usuario durante el registro, garantizando que no hay dos hashes idénticos aunque las contraseñas sean iguales.

BCrypt Algoritmo criptográfico de hash diseñado específicamente para contraseñas (Provos & Mazières, 1999). A diferencia de funciones de hash rápidas como SHA-256, BCrypt

es deliberadamente *lento* y adaptativo: incluye un factor de costo que puede incrementarse sin modificar la base de datos, lo que le permite “envejecer gracefully” mientras la potencia de cálculo aumenta. El factor de costo, junto con la sal aleatoria, hace computacionalmente inviable un ataque de fuerza bruta. En el proyecto, se utiliza en `UsuarioApplicationService` para cifrar las contraseñas de registro y autenticación.

Este glosario es una referencia viva del proyecto. Nuevos términos pueden agregarse conforme se expandan las funcionalidades y se incorporen nuevas técnicas de ML o trading.

B. Esquema de la base de datos

Este anexo describe la estructura de las nueve tablas que componen el esquema relacional del sistema, tal como queda definida en el repositorio del proyecto. Las decisiones de tipos reflejan los criterios descritos en el capítulo 6.4: `DECIMAL(18,8)` para todos los valores financieros, `BIGINT` para las marcas temporales expresadas en milisegundos de época, y la columna `eliminado` de tipo `BIT(1)` como mecanismo de borrado lógico uniforme. Las columnas de volumen cotizado emplean `DECIMAL(20,8)` para acomodar cifras de mayor magnitud sin pérdida de precisión.

Tabla usuario

Almacena las credenciales de acceso y el estado de cada cuenta registrada en la plataforma.

Cuadro B.1: Columnas de la tabla `usuario`.

Columna	Tipo	Descripción
<code>id</code>	<code>BIGINT</code>	Clave primaria autoincrementada
<code>nombre</code>	<code>VARCHAR(255)</code>	Nombre de usuario; único y obligatorio
<code>password_hash</code>	<code>VARCHAR(255)</code>	Contraseña cifrada con BCrypt
<code>fecha_creacion</code>	<code>DATETIME(6)</code>	Marca temporal de registro
<code>eliminado</code>	<code>BIT(1)</code>	Indicador de borrado lógico; valor por defecto 0

Tabla vela

Registra los datos OHLCV descargados de Binance para cada símbolo e intervalo temporal. La clave única compuesta por símbolo, intervalo y marca temporal de apertura garantiza la idempotencia de las inserciones masivas.

Cuadro B.2: Columnas de la tabla vela.

Columna	Tipo	Descripción
id	BIGINT	Clave primaria autoincrementada
symbol	VARCHAR(10)	Par de mercado (p. ej. BTCUSD)
time_interval	VARCHAR(10)	Marco temporal (p. ej. 1h, 4h)
open_time	BIGINT	Apertura de la vela en milisegundos de época
close_time	BIGINT	Cierre de la vela en milisegundos de época
open	DECIMAL(18,8)	Precio de apertura
high	DECIMAL(18,8)	Precio máximo
low	DECIMAL(18,8)	Precio mínimo
close	DECIMAL(18,8)	Precio de cierre
volume	DECIMAL(18,8)	Volumen negociado en activo base
quote_volume	DECIMAL(20,8)	Volumen negociado en activo cotizado
trades	INT	Número de transacciones en la vela
taker_base_volume	DECIMAL(18,8)	Volumen tomador en activo base
taker_quote_volume	DECIMAL(20,8)	Volumen tomador en activo cotizado
fecha_creacion	DATETIME(6)	Marca temporal de inserción
eliminado	BIT(1)	Indicador de borrado lógico

Tabla wallet

Representa una cartera digital perteneciente a un usuario. El doble saldo entre balance real y balance disponible impide que varias estrategias activas simultáneas puedan comprometer el mismo capital.

Tabla instancia_estrategia

Cada fila representa una configuración activa de ejecución de una estrategia, con su capital asignado, parámetros de riesgo y estado de ciclo de vida.

Tabla instancia_simbolos

Relación de los pares de mercado vigilados por una instancia de estrategia. Una misma instancia puede operar sobre varios símbolos simultáneamente.

Cuadro B.3: Columnas de la tabla `wallet`.

Columna	Tipo	Descripción
<code>id</code>	<code>BIGINT</code>	Clave primaria autoincrementada
<code>nombre</code>	<code>VARCHAR(255)</code>	Nombre descriptivo de la cartera
<code>usuario_id</code>	<code>BIGINT</code>	Referencia a <code>usuario(id)</code>
<code>balance_real</code>	<code>DECIMAL(18,8)</code>	Patrimonio total, incluyendo capital comprometido
<code>balance_disponible</code>	<code>DECIMAL(18,8)</code>	Capital libre para nuevas operaciones
<code>type</code>	<code>VARCHAR(50)</code>	Tipo de cartera (simulada o real)
<code>is_active</code>	<code>BIT(1)</code>	Indica si la cartera está operativa
<code>fecha_creacion</code>	<code>DATETIME(6)</code>	Marca temporal de creación
<code>eliminado</code>	<code>BIT(1)</code>	Indicador de borrado lógico

Tabla `posicion`

Registra cada operación de mercado individual, con el precio de entrada exacto y el margen comprometido. El campo `abierta` permite al motor evitar órdenes duplicadas sobre el mismo par mientras la posición sigue vigente.

Tabla `ledger_entry`

Constituye el libro mayor inmutable del sistema. Cada fila registra un movimiento de capital con su tipo, importe, saldo resultante y referencia descriptiva. Las entradas no se modifican ni eliminan físicamente, lo que permite reconstruir el estado financiero de cualquier cartera en cualquier instante histórico.

Tabla `indicador_tecnico`

Asociar cada vela almacenada con los valores de los indicadores técnicos calculados sobre ella por el motor Python. El campo `parametros` permite distinguir variantes del mismo tipo de indicador con periodos distintos.

Tabla `capital_reservado`

Desglosa por instancia de estrategia el capital bloqueado en cada cartera, permitiendo conciliar en todo momento los distintos estados del capital sin depender únicamente del libro mayor.

Cuadro B.4: Columnas de la tabla `instancia_estrategia`.

Columna	Tipo	Descripción
<code>id</code>	BIGINT	Clave primaria autoincrementada
<code>nombre_estrategia</code>	VARCHAR(255)	Identificador de la clase de estrategia
<code>wallet_asociada</code>	BIGINT	Referencia a la cartera digital
<code>timeframe</code>	VARCHAR(20)	Marco temporal de ejecución
<code>capital_asignado</code>	DECIMAL(18,8)	Capital total destinado a la instancia
<code>risk_per_trade</code>	DECIMAL(18,8)	Riesgo máximo por operación
<code>capital_reservado</code>	DECIMAL(18,8)	Capital bloqueado al activar la instancia
<code>capital_comprometido</code>	DECIMAL(18,8)	Capital en uso en posiciones abiertas
<code>riesgo_abierto</code>	DECIMAL(18,8)	Riesgo acumulado en posiciones vigentes
<code>es_real</code>	BIT(1)	Diferencia operativa real de simulada
<code>estado</code>	VARCHAR(50)	Estado del ciclo de vida (activa, pausada, terminada)
<code>fecha_creacion</code>	DATETIME(6)	Marca temporal de creación
<code>eliminado</code>	BIT(1)	Indicador de borrado lógico

Cuadro B.5: Columnas de la tabla `instancia_simbolos`.

Columna	Tipo	Descripción
<code>instancia_id</code>	BIGINT	Referencia a <code>instancia_estrategia(id)</code>
<code>simbolo</code>	VARCHAR(255)	Par de mercado supervisado (p. ej. ETHUSDT)

Cuadro B.6: Columnas de la tabla `posicion`.

Columna	Tipo	Descripción
<code>id</code>	BIGINT	Clave primaria autoincrementada
<code>simbolo</code>	VARCHAR(20)	Par de mercado de la operación
<code>precio_entrada</code>	DECIMAL(18,8)	Precio de entrada con precisión decimal completa
<code>margen_invertido</code>	DECIMAL(18,8)	Capital comprometido como margen
<code>abierta</code>	BIT(1)	Indica si la posición sigue activa
<code>instancia_id</code>	BIGINT	Referencia a <code>instancia_estrategia(id)</code>
<code>fecha_creacion</code>	DATETIME(6)	Marca temporal de apertura
<code>eliminado</code>	BIT(1)	Indicador de borrado lógico

Cuadro B.7: Columnas de la tabla `ledger_entry`.

Columna	Tipo	Descripción
<code>id</code>	BIGINT	Clave primaria autoincrementada
<code>wallet_id</code>	BIGINT	Cartera afectada por el movimiento
<code>estrategia_id</code>	BIGINT	Instancia de estrategia relacionada
<code>type</code>	VARCHAR(50)	Tipo de movimiento (véase sección 6.4)
<code>amount</code>	DECIMAL(18,8)	Importe del movimiento
<code>balance_after</code>	DECIMAL(18,8)	Saldo de la cartera tras el movimiento
<code>reference</code>	VARCHAR(255)	Descripción textual del concepto
<code>timestamp</code>	DATETIME(6)	Instante exacto del movimiento
<code>fecha_creacion</code>	DATETIME(6)	Marca temporal de inserción
<code>eliminado</code>	BIT(1)	Indicador de borrado lógico

Cuadro B.8: Columnas de la tabla `indicador_tecnico`.

Columna	Tipo	Descripción
<code>id</code>	BIGINT	Clave primaria autoincrementada
<code>vela_id</code>	BIGINT	Referencia a <code>vela(id)</code> ; obligatoria
<code>tipo</code>	VARCHAR(50)	Tipo de indicador (RSI, EMA, SMA, etc.)
<code>parametros</code>	VARCHAR(100)	Parámetros de cálculo (p. ej. periodo=14)
<code>valor</code>	DECIMAL(20,8)	Valor calculado del indicador
<code>fecha_creacion</code>	DATETIME(6)	Marca temporal de inserción
<code>eliminado</code>	BIT(1)	Indicador de borrado lógico

Cuadro B.9: Columnas de la tabla `capital_reservado`.

Columna	Tipo	Descripción
<code>id</code>	BIGINT	Clave primaria autoincrementada
<code>wallet_id</code>	BIGINT	Cartera a la que pertenece el registro
<code>estrategia_id</code>	BIGINT	Instancia de estrategia asociada
<code>reservado</code>	DECIMAL(18,8)	Capital bloqueado al activar la estrategia
<code>comprometido</code>	DECIMAL(18,8)	Capital actualmente en uso en posiciones
<code>riesgo_abierto</code>	DECIMAL(18,8)	Riesgo acumulado en posiciones vigentes
<code>fecha_creacion</code>	DATETIME(6)	Marca temporal de inserción
<code>eliminado</code>	BIT(1)	Indicador de borrado lógico

C. Referencia del protocolo IPC

Este anexo documenta el contrato técnico completo de la comunicación entre el orquestador Java y los motores Python, ampliando la descripción del protocolo incluida en la sección [6.4.2](#).

Estructura del mensaje

Todo intercambio sigue la estructura de *framing* binario definida en `IpMessagePackCodec` y `ipc_protocol.py`: un prefijo de cuatro bytes en orden de red que indica la longitud del cuerpo, seguido del cuerpo serializado en `MessagePack`. El cuerpo corresponde siempre a un objeto envoltente con los cuatro campos definidos en el registro `IpEnvelope`:

Cuadro C.1: Campos del objeto envoltente `IpEnvelope`.

Campo	Tipo	Descripción
<code>protocol_version</code>	String	Versión del protocolo; actualmente "1.0"
<code>message_type</code>	String	Código semántico de la operación (véase tabla C.2)
<code>correlation_id</code>	String	Identificador UUID único por petición
<code>payload</code>	objeto	Contenido específico de la operación

Tipos de mensaje

La enumeración `IpMessageType` define los veintiún códigos de operación reconocidos por los motores actuales. La tabla [C.2](#) los agrupa por dominio funcional.

Convención de códigos de salida

Complementando los tipos de mensaje, los motores Python comunican el resultado final de su ejecución mediante el código de retorno del proceso, que el orquestador Java interpreta para decidir si la operación fue exitosa o requiere tratamiento de error:

Cuadro C.2: Tipos de mensaje del protocolo IPC, agrupados por dominio.

Dominio	Tipo	Dirección
Descarga de datos	FETCH_REQUEST	Java → Python
	FETCH_CHUNK	Python → Java (respuesta parcial)
	FETCH_RESPONSE	Python → Java (fin de descarga)
Indicadores técnicos	INDICATORS_REQUEST	Java → Python
	INDICATORS_RESPONSE	Python → Java
Entrenamiento de modelos	TRAIN_REQUEST	Java → Python
	TRAIN_RESPONSE	Python → Java
Optimización de hiperparámetros	OPTIMIZE_REQUEST	Java → Python
	OPTIMIZE_RESPONSE	Python → Java
Inferencia	PREDICT_REQUEST	Java → Python
	PREDICT_RESPONSE	Python → Java
Simulación histórica	BACKTEST_REQUEST	Java → Python
	BACKTEST_RESPONSE	Python → Java
Trading en tiempo real	RT_SIGNAL	Python → Java (señal de orden)
	RT_LOG	Python → Java (línea de registro)
Control	ERROR	Python → Java

Cuadro C.3: Códigos de salida de los motores Python.

Código	Significado
0	Ejecución completada con éxito
1	Error de lógica no recuperable sin intervención del usuario (estrategia inválida, modelo no entrenado, etc.)
2	Error en los datos de entrada
3	Interrupción por tiempo de espera agotado o cierre externo

Comportamiento de lectura y escritura

La función de lectura del módulo `ipc_protocol.py` implementa tres niveles de compatibilidad para garantizar interoperabilidad durante transiciones de versión: en primer lugar intenta leer el mensaje como un envoltorio con prefijo de longitud de cuatro bytes (protocolo v1); si la longitud declarada no concuerda con el tamaño del flujo recibido, reintenta la deserialización directamente como `MessagePack` sin prefijo (modo legado); y como último recurso interpreta el contenido como texto JSON plano. La función de escritura construye siempre el envoltorio completo en formato v1, añadiendo un identificador de correlación único generado en tiempo de ejecución, de modo que todas las respuestas de todos los motores siguen el mismo formato con independencia de la operación solicitada.

D. Espacios de búsqueda de hiperparámetros

La Tabla D.1 recoge los rangos explorados por el motor de optimización Optuna para cada familia de modelos. Los límites de cada intervalo fueron determinados experimentalmente durante el desarrollo del proyecto a partir de valores habitualmente recomendados en la literatura (Chen & Guestrin, 2016; Goodfellow y col., 2016; Ke y col., 2017) y ajustados a las características de los conjuntos de datos históricos empleados.

Cuadro D.1: Rangos de búsqueda de hiperparámetros por familia de modelo

Modelo	Hiperparámetro	Rango	Tipo
XGBoost	Número de estimadores	[50, 500]	entero
XGBoost	Tasa de aprendizaje	[0,01 ; 0,30]	flotante (log)
XGBoost	Profundidad máxima	[3, 12]	entero
XGBoost	Submuestreo de filas	[0,6 ; 1,0]	flotante
XGBoost	Submuestreo de columnas	[0,6 ; 1,0]	flotante
XGBoost	Regularización L_1	[10^{-8} ; 10]	flotante (log)
XGBoost	Regularización L_2	[10^{-8} ; 10]	flotante (log)
LightGBM	Número de estimadores	[50, 500]	entero
LightGBM	Tasa de aprendizaje	[0,01 ; 0,30]	flotante (log)
LightGBM	Profundidad máxima	[3, 12]	entero
LightGBM	Número de hojas	[15, 127]	entero
LightGBM	Muestras mínimas por hoja	[5, 50]	entero
LightGBM	Submuestreo de filas	[0,6 ; 1,0]	flotante
LightGBM	Submuestreo de columnas	[0,6 ; 1,0]	flotante
Bosque aleatorio	Número de estimadores	[50, 500]	entero
Bosque aleatorio	Profundidad máxima	[3, 30]	entero
Bosque aleatorio	Muestras mínimas de partición	[2, 20]	entero
Bosque aleatorio	Muestras mínimas por hoja	[1, 10]	entero
Bosque aleatorio	Características por nodo	$\{\sqrt{p}, \log_2 p, \text{todas}\}$	categorico
Red neuronal	Épocas de entrenamiento	[20, 200]	entero
Red neuronal	Tamaño de lote	{32, 64, 128, 256}	categorico
Red neuronal	Tasa de aprendizaje	[10^{-4} ; 10^{-2}]	flotante (log)
Red neuronal	Tasa de abandono	[0,1 ; 0,5]	flotante
Red neuronal	Capas ocultas	[1, 3]	entero
Red neuronal	Neuronas por capa	{16, 32, 64, 128}	categorico

Bibliografía

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the Spring Joint Computer Conference*, 483-485.
- Apache Maven – Project Documentation. (s.f.). Consultado el 18 de marzo de 2026, desde <https://maven.apache.org/guides/index.html>
- Baur, D., Hong, K., & Lee, A. (2017). Bitcoin: Medium of Exchange or Speculative Assets? *Journal of International Financial Markets, Institutions and Money*, 54. <https://doi.org/10.1016/j.intfin.2017.12.004>
- Binance. (2024a). Binance API Rate Limits. Consultado el 18 de marzo de 2026, desde <https://developers.binance.com/docs/binance-spot-api-docs/rest-api/limits>
- Binance. (2024b). Binance Spot API Documentation: REST API Market Data Endpoints. Consultado el 18 de marzo de 2026, desde <https://developers.binance.com/docs/binance-spot-api-docs/rest-api/market-data-endpoints>
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32. <https://doi.org/10.1023/A:1010933404324>
- Chan, E. P. (2013). *Algorithmic Trading: Winning Strategies and Their Rationale*. John Wiley & Sons.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321-357. <https://doi.org/10.1613/jair.953>
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System [arXiv: 1603.02754]. *CoRR*. <http://arxiv.org/abs/1603.02754>
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273-297. <https://doi.org/10.1007/BF00994018>
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.
- Fama, E. F. (1970). Efficient Capital Markets: A Review of Theory and Empirical Work. *The Journal of Finance*, 25(2), 383-417.
- Feurer, M., & Hutter, F. (2019). Hyperparameter Optimization. En F. Hutter, L. Kotthoff & J. Vanschoren (Eds.), *Automated Machine Learning: Methods, Systems, Challenges* (pp. 3-33). Springer.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Freqtrade Development Team. (2023). Freqtrade: Free, open source crypto trading bot. <https://github.com/freqtrade/freqtrade>
- Freqtrade Team. (2026). Freqtrade Documentation: Strategy Customization. Consultado el 18 de febrero de 2026, desde <https://www.freqtrade.io/en/stable/>

- Furuhashi, S. (2013). MessagePack Specification. Consultado el 18 de marzo de 2026, desde <https://github.com/msgpack/msgpack/blob/master/spec.md>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.
- GitHub, Inc. (2024). GitHub Copilot: Your AI pair programmer. Consultado el 20 de abril de 2026, desde <https://github.com/features/copilot>
- Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java Concurrency in Practice*. Addison-Wesley.
- Goetz, B., & Pressler, R. (2023). JEP 444: Virtual Threads. Consultado el 18 de marzo de 2026, desde <https://openjdk.org/jeps/444>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Hasso, T., Pelster, M., & Breitmayer, B. (2019). Who trades cryptocurrencies, how do they trade it, and how do they perform? *Journal of Behavioral and Experimental Finance*, 23, 64-74. <https://doi.org/10.1016/j.jbef.2019.04.009>
- Hilpisch, Y. (2018). *Python for Finance: Mastering Data-Driven Finance* (2.^a ed.). O'Reilly Media, Inc.
- Hohpe, G., & Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley.
- Inmon, W. H. (2005). *Building the Data Warehouse* (4.^a ed.). Wiley Publishing.
- Katsiampa, P. (2017). Volatility estimation for Bitcoin: A comparison of GARCH models. *Economics Letters*, 158, 3-6. <https://doi.org/10.1016/j.econlet.2017.06.023>
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 3146-3154.
- Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley Publishing.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- López de Prado, M. (2018). *Advances in Financial Machine Learning*. John Wiley & Sons.
- Lusardi, A., & Mitchell, O. S. (2014). The Economic Importance of Financial Literacy: Theory and Evidence. *Journal of Economic Literature*, 52(1), 5-44. <https://doi.org/10.1257/jel.52.1.5>
- Martin, R. C. (2003). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall.
- Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy and Jupyter* (3.^a ed.). O'Reilly Media.
- Murphy, J. J. (1999). *Technical Analysis of the Financial Markets*. Penguin.
- Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Inc.
- Oracle Corporation. (2024a). MySQL 8.0 Reference Manual. Consultado el 18 de marzo de 2026, desde <https://dev.mysql.com/doc/refman/8.0/en/>
- Oracle Corporation. (2024b). MySQL 8.0 Reference Manual: InnoDB and the ACID Model. Consultado el 18 de marzo de 2026, desde <https://dev.mysql.com/doc/refman/8.0/en/mysql-acid.html>

- Oracle Corporation. (2024c). MySQL 8.0 Reference Manual: INSERT ON DUPLICATE KEY UPDATE. Consultado el 18 de marzo de 2026, desde <https://dev.mysql.com/doc/refman/8.0/en/insert-on-duplicate.html>
- Oracle Corporation. (2024d). MySQL 8.0 Reference Manual: Optimization of INSERT Statements. Consultado el 18 de marzo de 2026, desde <https://dev.mysql.com/doc/refman/8.0/en/insert-optimization.html>
- OWASP. (2024). Password Storage Cheat Sheet. Consultado el 18 de marzo de 2026, desde https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- Pardo, R. (2008). *The Evaluation and Optimization of Trading Strategies* (2.^a ed.). John Wiley & Sons.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- Project Lombok: Features. (s.f.). Consultado el 18 de marzo de 2026, desde <https://projectlombok.org/features/>
- Provos, N., & Mazières, D. (1999). A Future-Adaptable Password Scheme. *Proceedings of the USENIX Annual Technical Conference*, 32-32.
- Python Software Foundation. (2024). Python 3.11 Documentation: `concurrent.futures`. Consultado el 18 de marzo de 2026, desde <https://docs.python.org/3/library/concurrent.futures.html>
- Requests: HTTP for Humans. (s.f.). Consultado el 18 de marzo de 2026, desde <https://docs.python-requests.org/en/latest/>
- Richards, M. (2022). *Software Architecture Patterns* (2.^a ed.). O'Reilly Media, Inc.
- SonarSource S.A. (2024). SonarLint: Clean Code in your IDE. Consultado el 20 de abril de 2026, desde <https://www.sonarsource.com/products/sonarlint/>
- Spring Team. (2024). Spring Data JPA Reference Documentation. Consultado el 18 de marzo de 2026, desde <https://docs.spring.io/spring-data/jpa/reference/index.html>
- Stevens, W. R., & Rago, S. A. (2005). *Advanced Programming in the UNIX Environment* (2.^a ed.). Addison-Wesley.
- Treleaven, P., Galas, M., & Lalchand, V. (2013). Algorithmic Trading Review. *Communications of the ACM*, 56(11), 76-85. <https://doi.org/10.1145/2500117>
- twopirllc. (2024). Pandas TA: Technical Analysis Indicators for Pandas. Consultado el 18 de marzo de 2026, desde <https://github.com/twopirllc/pandas-ta>
- VMware Tanzu & Spring Team. (2024). Spring Boot Reference Documentation [Versión 3.x]. Consultado el 18 de marzo de 2026, desde <https://docs.spring.io/spring-boot/reference/index.html>
- Wooldridge, B. (2024). HikariCP: A solid, high-performance JDBC connection pool. Consultado el 18 de marzo de 2026, desde <https://github.com/brettwooldridge/HikariCP>